Vietnam National University Ho Chi Minh City
Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering

– INTERNSHIP 2 REPORT –

Cost-Aware Validation Scheduling and Selected Hints for LLM-Based Program Repair

Council: Computer Science and Engineering
Instructor: Dr. Le Xuan Bach, CSE-HCMUT
Student: Nguyen Thai Thanh Dat
Student ID: 2570387

Ho Chi Minh City, November 2026

Date: _____

Dr. Le Xuan Bach, CSE-HCMUT
Faculty of Computer Science and Engineering

Acknowledgements

I would like to thank the lecturers of the Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (HCMUT), for the foundational and specialised teaching that made this work possible.

I am especially grateful to my instructor, Dr. Le Xuan Bach, CSE-HCMUT, for the guidance that shaped the scope of this work. Two pieces of that guidance did most of the work. The first was the insistence that a research question be stated together with the outcome that would *refute* it; applying that test removed several items from an earlier list that turned out to be names for groups of experiments rather than questions. The second was the instruction to measure the resource the system actually spends rather than the one the literature happens to report, which is why Chapter 5 reports CPU seconds and test executions side by side instead of either alone.

I also thank my colleagues for the discussions that sharpened the central distinction this work is built on: that a repair loop learns two different things on every iteration — which test is worth running, and which repair idea is already dead — and that both are usually discarded.

This document reports work in progress. The implementation and its unit tests are complete; the 100-task pilot is running at the time of writing, and Chapter 6 marks every number that depends on it. Any remaining errors are my own.

Abstract

A program repair tool built on a large language model (LLM) works in a loop. It asks the model for a patch, runs the program's tests to see whether the patch is wrong, and, if it is, feeds the failure back to the model and asks again. The loop repeats until a patch passes every test or until the attempt budget runs out.

Two things make this loop expensive. First, running the tests is the step that is repeated most often and costs the most: every candidate patch must be built and executed before anyone can tell whether it works. Second, the model keeps proposing patches it has already tried, and each repeat still has to be tested. Both problems have the same cause. The loop produces useful information on every iteration — which test rejected which patch, how long that test took, and which repair ideas have already failed — and then throws almost all of it away.

This report describes CASTER, a repair loop that keeps that information in a small per-test record and spends it in two ways. **Cost-aware validation scheduling** combines how often a test has rejected patches with how long that test takes to run, and uses the combination to decide which test to run first, so a bad patch is rejected as cheaply as possible. **Selected hints** replace the usual practice of pasting the whole conversation history into the prompt with a small, freshly chosen set of concrete facts that point the model away from ideas already shown to fail. The two mechanisms feed each other: the scheduler produces the failure information the hints are selected from, and better hints mean fewer wasted patches arriving at the expensive validation step.

Because CASTER works only at the level of *which test runs first* and *what goes into the prompt*, it is compatible with the two other families of cost reduction in the literature: systems that make each test execution faster, and learned verifiers that skip execution for some candidates altogether.

Status of the evaluation. The system is implemented and unit-tested. The evaluation reported here is a pilot on 100 Python tasks from ConDefects [1], a benchmark built so that its faults post-date the training cut-off of commonly used models. The pilot compares CASTER against the order the benchmark ships, an uninformed random order, and the published fault-recorded prioritiser FRTP [2], measuring CPU seconds as the primary cost unit and test executions alongside it. Chapter 6 is laid out for those results and every number in it is marked as pending until the pilot finishes. No number in this report is presented as a final result at corpus scale.

Contents

Acknowledgements **iv**

Abstract **v**

1 Introduction **1**
1.1 The Repair Loop 1
1.2 Two Problems with Current Practice 1
1.3 The Information the Loop Discards 2
1.4 What Existing Solutions Miss 2
1.5 This Work: Caster 3
1.6 Contributions 3
1.7 Structure of This Report 4

2 Related Works **5**
2.1 Reducing Patch Validation Cost 5
2.2 Steering Generation in LLM-Based Repair 7
2.3 Context Selection for LLM-Based Repair 8
2.4 Summary: Where This Work Sits 8

3 Background **11**
3.1 Iterative LLM-Based Repair 11
3.1.1 The Objects 11
3.1.2 The Loop as a Formal Object 11
3.1.3 The Two Bills 12
3.2 Validation Cost and Test Ordering 12
3.2.1 What a Session Costs 12
3.2.2 Why Executions and Seconds Are Not Interchangeable 13
3.2.3 Order Changes the Cost but Not the Answer 13
3.2.4 Test-Case Prioritisation 14
3.2.5 What Frtp Records and What It Discards 14
3.3 Repair Hypotheses and History in the Prompt 15
3.3.1 Patches Versus Hypotheses 15
3.3.2 Why Showing the History Is Not Enough 16

3.3.3 What a Hint Has to Be 16
3.4 Benchmark Background 17

4 Methodology **18**
4.1 Overview 18
4.1.1 Requirements the Design Must Satisfy 19
4.1.2 What the Mechanism Costs 20
4.2 The Repair History Store 21
4.2.1 What Is Stored 21
4.2.2 The Two Derived Estimates 21
4.2.3 Update Rules 22
4.3 Cost-Aware Validation Scheduling 22
4.3.1 The Ranking Rule 22
4.3.2 Correcting a Marginal Rate 23
4.4 Selected Hints 25
4.4.1 What a Hint Is 25
4.4.2 How Hints Are Selected 26
4.4.3 The Disclosure Dial 27
4.4.4 Why This Is Not a Transcript 28

5 Experiments **29**
5.1 Research Questions 29
5.2 Benchmark: ConDefects Python 30
5.2.1 Corpus, and the 100-Task Pilot Drawn From It 30
5.2.2 Pilot Scale and Why It Is a Pilot 32
5.2.3 Screens Are Bands, Decided After the Run 32
5.3 Baselines and Configurations 33
5.3.1 The Two Blocks 33
5.3.2 Block A — Scheduling Configurations 34
5.3.3 Block B — Prompt Configurations 35
5.3.4 The Factorial for RQ3 35
5.3.5 Parameter Sweeps 36
5.4 Metrics 36
5.5 The Ablation Ladder 37
5.6 Implementation and Measurement 38
5.6.1 The Tool 38
5.6.2 How Cost Is Measured 38
5.6.3 Reproducibility 39

5.7 Statistical Analysis 39

6 Results **41**
6.1 Corpus and Measurement Summary 41
6.2 RQ1: Effectiveness of Cost-Aware Scheduling 41
6.2.1 Head-to-Head 41
6.2.2 Results by Band 43
6.2.3 Learning Curve 43
6.2.4 Soundness Check 44
6.3 RQ2: Effectiveness of Selected Hints 44
6.4 RQ3: Combined Effect 45
6.5 RQ4: Ablation and Sensitivity 46
6.5.1 Which Term Carries the Scheduling Effect 46
6.5.2 How Much to Divide by Cost 46
6.5.3 The Hint Dose 48
6.5.4 Sensitivity to Design Choices 48
6.6 Summary of Findings 48

7 Discussion **49**
7.1 When Caster Does Not Help 49
7.1.1 C3 — A Task Too Easy for the Proposer 50
7.1.2 How the Three Are Reported 50
7.2 Reporting CPU Rather Than Executions 51
7.3 Combining with Acceleration and Verifiers 52
7.4 The Cost of Adopting It 53
7.5 What the Pilot Can and Cannot Settle 54

8 Threats to Validity **55**
8.1 Internal Validity 55
8.2 External Validity 56
8.3 Construct Validity 56
8.4 Conclusion Validity 57

9 Conclusion and Future Work **58**
9.1 Conclusion 58
9.2 Future Work 59

References **61**

List of Figures

1.1 The iterative generate–validate–refine loop used by most LLM-based repair systems. Validation is the step inside the cycle, so its cost is paid once per candidate patch rather than once per bug. 1

2.1 The three directions in validation-cost reduction 6

3.1 Why a count of rejections is not enough 15

4.1 One round of the CASTER loop. The repair history store is written by validation and read by both the scheduler and the hint selector, which is why the two mechanisms are one system rather than two independent features. 18

6.1 Validation CPU per round. Round 1 is the same for every configuration; what happens afterwards is the store filling in. An incumbent line that stays flat is the loop never learning anything about its pool. 43

6.2 Duplicate rate over the course of a session. The gap between configurations should widen with round number, because that is when the history exists to be selected from. 45

6.3 Cost-exponent sweep. An interior minimum would mean the full cost division over-corrects, which is what a noisy online denominator predicts; a minimum at an endpoint would mean one of the two published positions is already right. 46

6.4 The disclosure dose. A curve that is flat across both axes means there is no dose and the disclosure dial does not exist; a leakage rate that climbs with disclosure means the model is beginning to satisfy the tests it was shown rather than repairing the fault. 47

List of Tables

2.1 The nearest neighbours on the three design dimensions 10

3.1 Terms used in this report. 12

4.1 Per-round data flow. 19
4.2 What the mechanism costs to run 20
4.3 Disclosure levels, from least to most revealed. 27

5.1 Corpus admission criteria. Applied once, before treatment. 31
5.2 Pilot configuration. 32
5.3 Band axes, declared before the run 33
5.4 Block A: configurations that differ only in test order. "Answers" names the research question each one serves. 34
5.5 Block B: configurations that differ in what the prompt carries. All use the proposed scheduler unless the name says otherwise. 35
5.6 The 2×2 design behind RQ3. Each cell is a configuration run on the same 100 tasks $\times$ 3 seeds. 35
5.7 Validation-side metrics (RQ1, RQ3, RQ4). 36
5.8 Generation-side metrics (RQ2, RQ3, RQ4). 37
5.9 The ablation ladder for RQ4. One variable per step. 38

6.1 Admission funnel: how the 100 pilot tasks were obtained. 41
6.2 The two distributions that decide whether cost-aware scheduling has anything to work with. 42
6.3 Validation cost per configuration (RQ1) 42
6.4 Whole corpus against the three-condition band 43
6.5 Soundness: every Block A configuration must accept exactly the same patches on exactly the same rounds (Definition 3.2). A difference here is an implementation bug, not a finding. 44
6.6 Generation-side results (RQ2) 44
6.7 Paired contrasts for RQ2, on the duplicate rate. 44
6.8 The 2×2 design of Table 5.6, measured. The combined cell is the full system. 45

6.9 Interaction. "Additive prediction" is what the combined cell would cost if the two mechanisms did not interact; the residual is the interaction term. 45
6.10 The ablation ladder of Table 5.9, measured. One variable changes per row. Exploratory: intervals are reported without correction and no claim rests on a single row. 46
6.11 Hint dose: number of hints crossed with disclosure level. "Leakage" is the share of rejected patches whose rejecting tests are disjoint from what the prompt showed — the patch satisfied everything it was told and died elsewhere. 47
6.12 Sensitivity configurations, each paired against `sched`. These belong to threats to validity rather than to the contribution: the question is whether RQ1's answer survives choices made for other reasons. 47

7.1 The three conditions for the method to pay off 49

Chapter 1

Introduction

1.1 The Repair Loop

A *bug* is a place where a program does the wrong thing. A *test case* is one concrete check: an input, and the output the program is supposed to produce for it. A *test suite* is a collection of such checks, and a *patch* is an edit to the program that is meant to make the failing checks pass. *Automated program repair* (APR) is the task of producing such a patch without a human writing it [3].

In APR based on large language models (LLMs), one design dominates: an iterative *generate–validate–refine* loop. Given a buggy program and its failing tests, the LLM generates a candidate patch for the suspicious code. The patch is then *validated* by running the test suite against it. If validation fails, the resulting feedback — the failing test cases, the error message, or the stack trace — is fed back to the model so that it can refine the patch, and the loop repeats. It stops when a patch passes all tests, which makes it *plausible*, or when the iteration budget is exhausted. Figure 1.1 shows the arrangement.

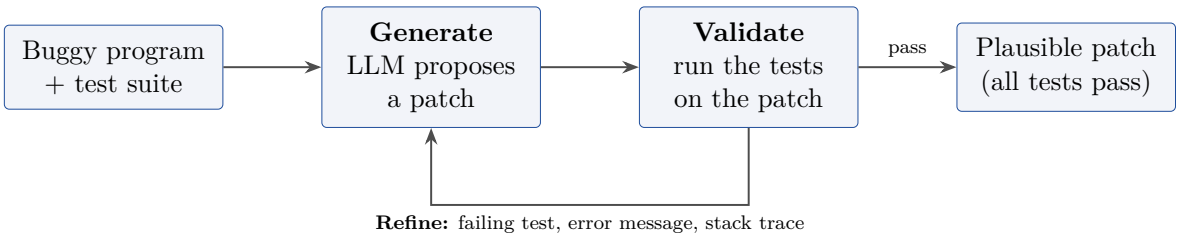


Figure 1.1: The iterative generate–validate–refine loop used by most LLM-based repair systems. Validation is the step inside the cycle, so its cost is paid once per candidate patch rather than once per bug.

1.2 Two Problems with Current Practice

Current practice reveals two issues with this loop.

Validation is the most repeated and the most expensive step. Each candidate patch must be built and executed against the test suite before its correctness can be judged, and this check is repeated for every candidate the LLM produces. Lin et al. [4] analysed 7,745 agent traces from SWE-bench leaderboard submissions and found that every agent and model examined relies on code execution, averaging 8.8 test runs per task, with frequency varying widely from 2 to 19 runs per task across agents and models. Each of these runs is costly: RepairAgent [5] spends 99 % of its total repair time on tool executions, mostly running tests, with a median repair time of about 920 seconds per bug. Validation therefore dominates the end-to-end cost of LLM-based repair, and every candidate that reaches this step adds directly to that cost.

LLMs regenerate patches they have already tried. When candidates are sampled independently, the LLM is unaware of its previously generated patches and may produce many repeated or similar candidates [6]. Exposing this history does not eliminate the problem: ChatRepair [6] keeps earlier incorrect patches and their test-failure information in the conversation, yet Chen et al. [7] found that duplicate patches still constituted 65 % of all patches it generated during plausible patch generation on Defects4J bugs. Feedback-driven refinement does not fully resolve the problem either: when the underlying repair hypothesis is flawed, refinement tends to produce only superficial variants of the same incorrect idea [8]. Since every such candidate must still be validated, this redundancy directly inflates the cost of the already expensive validation step.

1.3 The Information the Loop Discards

Both issues stem from the same root: the repair loop produces valuable information but makes little use of it. Every validation reveals which tests reject which candidates and at what cost, and every generation reveals which repair hypotheses have already been explored. This history offers two levers. It can **schedule validation**, so that each candidate is judged as cheaply as possible, and it can **steer generation**, so that the LLM is directed away from directions that have already failed.

1.4 What Existing Solutions Miss

Prior work reduces validation cost by making each test run cheaper [9, 10], by skipping execution with learned verifiers [11], or by scheduling which tests to run [2, 12]. Our work targets scheduling, which is orthogonal to execution acceleration. Existing sched-

ulers, however, optimise the *number* of test executions rather than their *cost*. FRTP [2], for instance, first runs the tests that failed earlier candidates, regardless of how long each one takes, even though long-running tests are what make validation expensive.

On the generation side, prior work explores the repair space through tree search [13], retains earlier attempts in memory [6, 14], reasons explicitly about bug hypotheses [15, 16], or transfers experience across bugs [17]. Yet *how* history is presented matters as much as *whether* it is presented. As noted above, CHATREPAIR still produces mostly duplicate patches despite seeing its own history [7]. Parasaram et al. [18] further show that adding more facts to the prompt can hurt repair, but their selection is made once, over static facts, before generation. How the evolving history of a single repair session should be condensed, at each iteration, into guidance toward untried repair directions remains open.

1.5 This Work: Caster

To fill these gaps, this work proposes CASTER, an LLM-based repair approach that turns the repair history into two complementary mechanisms.

Cost-aware validation scheduling combines the failure history of each test with its execution cost to decide which tests to run first, so that unpromising patches are rejected with minimal execution time.

Selected hints replace the accumulation of raw history in the prompt with a small, iteration-specific set of hints, distilled from previously tried patches and their failures, that steer the LLM away from repair hypotheses already shown to fail.

The two mechanisms reinforce each other: scheduling produces the failure information from which hints are selected, and better-steered generation reduces the number of redundant candidates that reach validation in the first place. Because CASTER operates at the level of scheduling and prompting, it can be combined with execution-acceleration frameworks [9, 10] and execution-free verifiers [11].

1.6 Contributions

1. **A per-test repair history store.** One record per test case, updated as the loop runs, holding how many times the test has been run, how many of those runs rejected a patch, and how much CPU time those runs took (Section 4.2). The same record serves both mechanisms, which is why they ship together rather than as two independent features.

2. **Cost-aware validation scheduling.** A scheduling rule that ranks tests by an

estimated rejection rate divided by an estimated cost, so that the loop prefers tests that are likely to reject a patch *and* cheap to run (Section 4.3). Existing repair-loop schedulers rank on a count of past rejections only.

3. **Selected hints.** A prompt contract that, at every iteration, selects a small number of concrete facts from the history and renders them as requirements the next patch must satisfy, rather than appending the full transcript (Section 4.4).

4. **A mechanism cheap enough to adopt.** Both mechanisms run on one record of three numbers per test case, updated in constant time (Section 4.1.2). Nothing is trained, no profiling pass is needed, and neither mechanism adds a single program execution or model call — the scheduler reorders runs the loop was already making, and hint selection only reads what validation has already recorded. This matters for comparison as well as for deployment: the other two families of validation-cost reduction require a language-runtime integration or a trained verifier before they return anything (Section 7.4).

5. **An evaluation protocol that reports the resource actually spent.** CPU seconds as the primary unit with test executions reported alongside, paired per task, with interval estimates from a bootstrap over tasks (Chapter 5). The two units can disagree, and a result reported in executions alone does not transfer to a deployment that pays for time.

1.7 Structure of This Report

Chapter 2 surveys related work in three groups: reducing patch validation cost, steering generation, and context selection. Chapter 3 builds the necessary background from first principles. Chapter 4 describes Caster. Chapter 5 states the research questions and the protocol for the 100-task pilot, and Chapter 6 reports the results. Chapters 7–9 discuss when the approach does and does not help, state the threats to validity, and conclude.

Chapter 2

Related Works

Work related to this report falls into three groups. The first reduces the cost of checking a candidate patch. The second helps the model explore the repair space instead of repeating itself. The third asks what should go into the prompt at all. CASTER touches all three: it schedules validation, it steers generation, and it does so by selecting a small amount of context rather than accumulating all of it.

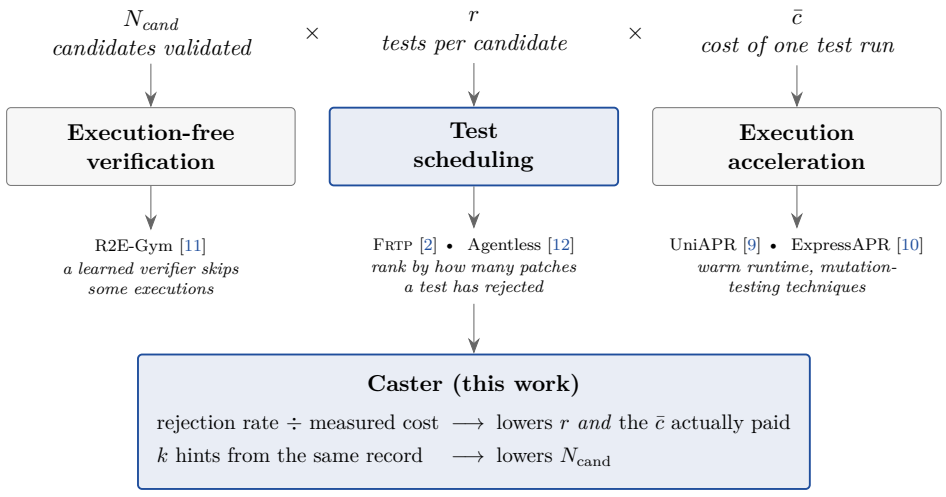
2.1 Reducing Patch Validation Cost

Patch validation has long been recognised as a bottleneck of test-based repair, and prior work reduces its cost along three largely complementary axes. The three are easiest to tell apart by asking which part of the bill each one attacks. Writing the total validation cost of a repair session as a product of three factors,

$$\text{validation cost} \approx \underbrace{N_{\text{cand}}}_{\substack{\text{candidates that}\\ \text{reach validation}}} \times \underbrace{r}_{\substack{\text{tests run}\\ \text{per candidate}}} \times \underbrace{\bar{c}}_{\substack{\text{cost of one}\\ \text{test run}}}, \tag{2.1}$$

each family reduces a different factor, and none of them reduces another family's. Figure 2.1 places the surveyed systems on that decomposition and marks where this work sits.

Execution acceleration makes each test run cheaper. Because patch validation repeatedly runs the same test suite on slightly different program variants, it is closely related to mutation analysis, and several works adapt acceleration techniques from that field. UNIAPR [9] validates patches on the fly within a single JVM, reloading only the patched classes through the HotSwap mechanism and resetting JVM state between patches to avoid interference. EXPRESSAPR [10] identifies five classes of mutation-testing acceleration techniques suitable for patch validation and integrates them into a general-purpose validator for Java, accelerating validation by 137.1× over plain validation and 8.8× over the prior state of the art. These techniques reduce the cost of each execution but leave unchanged *which* tests are executed and *in which*



The three families multiply rather than compete: a deployment can run an accelerated executor, a cost-aware schedule on top of it, and a verifier in front of both.

Figure 2.1: The three directions in reducing patch validation cost, placed on the factors of Equation (2.1). This work sits in test scheduling, and is the only entry we found there whose ranking reads the cost of a test as well as its record of rejecting patches.

order, and they are tied to a specific language runtime.

Execution-free verification avoids running tests for some candidates by assessing them with a learned model. In repository-level repair, R2E-GYM [11] compares execution-based and execution-free verifiers and finds their strengths complementary. Test-based verifiers often fail to distinguish among candidates, while execution-free verifiers are biased and tend to rely on stylistic features. Combining the two yields substantially higher performance than either alone. Such verifiers are therefore best seen as a complement to test execution rather than a replacement for it.

Test scheduling decides which tests to run and in which order. FRTP [2] records the tests that failed earlier candidate patches during the repair process and executes them first, so that invalid patches are rejected after fewer test executions. Because this information comes from the repair process itself, FRTP incurs only trivial overhead. Its implementation, TRPAUTOREPAIR, performed at least as well as GENPROG in success rate while significantly improving repair efficiency, on 15 of 16 real-world bugs. In LLM-based repair, AGENTLESS [12] first records which existing tests pass on the original program and asks the LLM to select those relevant to the edited code. It then rejects any candidate that causes a selected test to fail.

These schedulers optimise the *number* of test executions rather than their *cost*. In regression testing, by contrast, it has long been recognised that test costs vary and

that prioritisation should account for them [19]. In patch validation this distinction matters: a test that rejects many candidates but takes minutes to run may be a worse first choice than a slightly less discriminating test that finishes in milliseconds.

Our approach departs from these schedulers by scheduling tests using both their failure history within the repair session and their execution cost. Because it operates purely at the scheduling level, it can be combined with execution acceleration [9, 10] and execution-free verification [11].

2.2 Steering Generation in LLM-Based Repair

A second line of work aims to help LLMs explore the repair space more effectively instead of repeatedly sampling similar candidates.

Search-based approaches organise repair attempts into an explicit search structure. SWE-Search [13] integrates Monte Carlo Tree Search into a software agent and uses a hybrid value function, in which the LLM provides both numerical estimates and qualitative assessments, to balance exploration and exploitation. This yields a 23 % relative improvement on SWE-bench over agents without search. Search broadens exploration, but every additional branch produces candidates that must be validated, which increases the cost this work aims to reduce.

In-loop memory approaches retain information from earlier attempts on the same bug. ChatRepair [6] keeps previously generated incorrect patches and their test-failure information in the conversation so that the LLM avoids repeating them. Chen et al. [7], however, found that duplicate patches still constituted 65 % of the patches ChatRepair generated during plausible patch generation on Defects4J. Moreover, when the underlying repair hypothesis is flawed, feedback-driven refinement tends to produce only superficial variants of the same idea [8]. Outside program repair, Reflexion [14] lets language agents reflect verbally on feedback from failed trials and stores these reflections in an episodic memory that conditions subsequent attempts. In both cases, the history is accumulated or summarised *as it arrives* rather than selected according to what the next iteration needs.

Hypothesis-driven approaches make the LLM reason explicitly about the cause of a bug before or while patching it. AutoSD [15] prompts the LLM to formulate hypotheses about the bug, together with debugger experiments that test them. It executes these experiments and lets the LLM judge whether each hypothesis holds

before generating a fix. ThinkRepair [16] first builds a knowledge pool of chains of thought that led to validated fixes, then repairs a new bug by selecting examples from this pool for few-shot prompting, optionally refining the patch with test feedback. These approaches improve the quality of individual hypotheses but do not explicitly track which hypotheses have already been refuted within a repair session.

Experience-driven approaches transfer knowledge across bugs. SWE-Exp [17] observes that memoryless agents redundantly re-explore failed trajectories. It addresses this with an experience bank that captures both successful and failed repair attempts and distills reusable knowledge at multiple levels, from problem comprehension to concrete code changes. Such experience is retrieved from *other* bugs, whereas the history most informative for avoiding repeated failures is often the one produced on the current bug.

In contrast, we do not accumulate or summarise the session history; at each iteration Caster selects a small set of hints from it, ranked by the per-test rejection statistics that its validation scheduler already maintains, so that the LLM is directed by the tests that most often refute candidates and away from the hypotheses they have refuted. Unlike search-based approaches, it does so without generating additional candidates.

2.3 Context Selection for LLM-Based Repair

The effectiveness of LLM-based repair depends heavily on what information is included in the prompt. Parasaram et al. [18] systematically studied the bug-related facts that can be provided to an LLM, such as stack traces, test code, and repository context. They found that while each type of fact helps fix some bugs, repair effectiveness is non-monotonic in the number of facts included, so overly rich prompts degrade performance, and no single fact set is optimal across bugs. Their bug-specific fact selector repaired 88 of 157 bugs, 17 % above the best configuration. This selection, however, operates on *static* facts about the bug and is performed once, before generation begins.

Caster applies the same principle — that less but well-chosen context is better — to the *dynamic* history of an iterative repair loop. It reselects a small set of hints at every iteration as new patches are tried and new failures are observed.

2.4 Summary: Where This Work Sits

Figure 2.1 separated the surveyed work by *which part of the bill* it reduces. This section asks a narrower question about the scheduling family and its neighbours: of the design

decisions CASTER makes, which ones has someone already made?

Three dimensions distinguish it:

D1 Does the system decide *which test to run first*?

D2 Does the *measured cost* of a test enter that decision?

D3 Does the system *choose* what evidence enters the prompt, rather than including all of it or none of it?

Each dimension is satisfied by published work on its own. What we found no occupant for is the combination: Table 2.1 lists the systems that hold at least one of the three and says, for each, what is still different here. Two things about how to read it. Blank cells mean we are not aware of a system in that position, to the best of our knowledge, not that none exists — the table is a claim about the literature we surveyed. And D2 is only meaningful alongside D1, since the cost of a test can enter a ranking only if there is a ranking; there is no D2-without-D1 to look for.

Only systems holding at least one of D1–D3 appear. Those that reduce a different factor of the validation bill — UNIAPR, EXPRESSAPR, R2E-GYM — and those that change the search rather than the evidence — SWE-SEARCH, SWE-EXP, THINKREPAIR — are placed in Figure 2.1 instead, because scoring them here would produce a row of dashes and compare nothing.

Three gaps follow, and together they are what this work adds. **(1)** Repair-loop schedulers ignore cost: the one line of work that puts cost in the ranking [19] comes from regression testing and assumes the cost is known in advance. **(2)** A count of rejections discards the evidence of *not* rejecting, since a test run ten times that rejected nothing and a test never run both score zero; a rate is the first quantity that reads the denominator. **(3)** Session history is accumulated rather than selected — fact selection [18] does select, but once, over facts fixed before the first candidate is drawn.

Table 2.1: The systems that occupy at least one of D1–D3, and what still separates each from this work. ✓ is full, ○ is partial, blank is absent.

System	D1	D2	D3	What is still different here
Additional greedy [20]	✓			Ranks by coverage of the program, a static property, not by anything the repair loop observes
FRTP [2]	✓			Ranks on a *count* of rejections: no denominator, so it cannot tell an unrun test from one that never rejects; and no cost term
AGENTLESS [12]	✓			The model picks which tests look relevant; nothing records what actually rejected what, and nothing is timed
Cost-cognizant TCP [19]	✓	✓		Regression testing, not repair. Costs are supplied before the run rather than measured inside it, and no evidence is returned to a generator
Fact selection [18]			✓	Selects *static* facts about the bug, once, before generation begins. Says nothing about validation, and does not reselect as the session produces new failures
CHATREPAIR [6], REFLEXION [14]			○	History reaches the prompt, but it is accumulated rather than selected, so it grows with the session and dilutes the few facts that matter
Caster (this work)	✓	✓	✓	One per-test record drives both the schedule and the prompt

Chapter 3

Background

This chapter builds up the objects the rest of the report uses, starting from plain definitions. Section 3.1 defines the repair loop precisely. Section 3.2 explains what validation costs, why the order in which tests run changes that cost, and why counting test runs is not the same as measuring time. Section 3.3 explains what a repair hypothesis is and why putting the whole history in the prompt does not stop the model repeating one.

3.1 Iterative LLM-Based Repair

3.1.1 The Objects

Table 3.1 lists every term this report uses in a technical sense. Each is defined once here and used consistently afterwards.

3.1.2 The Loop as a Formal Object

Definition 3.1 (Repair session)**.** A repair session with budget B on a faulty program s_0 and a test pool P is the following sequence, for $t = 1, 2, \ldots, B$:

1. The LLM draws a candidate patch $s_t \sim \pi(\cdot \mid \mathcal{H}_{t-1})$, where $\mathcal{H}_{t-1}$ is whatever the loop has chosen to put in the prompt from rounds $1, \ldots, t-1$.
2. The *validator* runs s_t against test cases from P in some order σ_t, stopping at the first test case that rejects s_t.
3. If no test case rejects s_t, the session *accepts* s_t and stops. Otherwise the rejecting test case, together with whatever else the loop chooses to carry, becomes part of $\mathcal{H}_t$, and the next round begins.

Two choices are left open by Definition 3.1, and they are exactly the two levers of this work. The order σ_t is free, and nothing in the definition says it must be the order the benchmark happens to ship. The contents of $\mathcal{H}_t$ are free, and nothing says they must be the full transcript.

Table 3.1: Terms used in this report.

Term	Meaning in this report
Faulty program s_0	The program that contains the bug.
Test case x	One input together with the output the program is expected to produce for it.
Test pool P	The finite set of test cases available for one task, $P = \{x_1, \ldots, x_n\}$.
Reject (or refute)	A test case *rejects* a program when the program's output on that input differs from the expected output. One rejecting test is enough to prove the program wrong.
Candidate patch s	A modified program produced by the LLM in one iteration.
Round (or iteration)	One pass of generate → validate → refine.
Repair session	All rounds spent on one task, up to the budget B.
Budget B	The maximum number of rounds, that is, the maximum number of candidate patches, allowed per task.
Scan	One validation: running a candidate against test cases from P until one of them rejects it, or until all have passed.
Plausible patch	A candidate that no test case in P rejects. "Plausible" rather than "correct" because passing a finite test pool is weaker than being right [21].
Test-run cost $c(x)$	The CPU time consumed by running one candidate on one test case x.
Repair hypothesis	The idea behind a patch: what the model believes is wrong and how it intends to fix it. Many different patches can express one hypothesis.

3.1.3 The Two Bills

A repair session spends two different resources, and keeping them apart matters because optimising one can increase the other.

- **The generation bill**, paid in model tokens and therefore in money or in GPU time. It grows with the number of rounds and with the size of the prompt.
- **The validation bill**, paid in CPU seconds spent executing candidate programs. It grows with the number of rounds, with how many test cases each scan runs, and with how expensive those particular test cases are.

This report targets the validation bill with its scheduler and the generation bill with its hints, and reports both.

3.2 Validation Cost and Test Ordering

3.2.1 What a Session Costs

Write $c(x)$ for the CPU cost of running one candidate on test case x, and σ_t for the order the validator uses in round t, so that $\sigma_t(j)$ is the j-th test case it runs. The

validation cost of a session is

$$C \;=\; \sum_{t=1}^{T} \sum_{j=1}^{r_t} c(\sigma_t(j))\,, \qquad r_t \;=\; \min\{\, j \;:\; \sigma_t(j) \text{ rejects } s_t \,\}, \tag{3.1}$$

where T is the number of rounds actually run and $r_t = |P|$ when nothing rejects s_t. Two observations about Equation (3.1) drive the whole design.

First, σ_t is the only term the validator controls. The candidate comes from the model, the pool comes from the benchmark, and the per-test costs are properties of the programs; the order is a free choice made on every round.

Second, minimising the *number* of test runs $\sum_t r_t$ is the same objective as minimising C **only if** c **is constant**. When test costs differ, a schedule can run more tests and still finish sooner, or run fewer tests and take longer. The two quantities can therefore rank two schedules in opposite orders.

3.2.2 Why Executions and Seconds Are Not Interchangeable

The test-prioritisation literature usually reports the number of test executions rather than elapsed time, and does so for a defensible reason. The largest study of patch validation to date measures executions precisely because elapsed time is unstable across machines and configurations, where an execution count reproduces [22]. That choice is sound when executions are interchangeable — when running any one test costs about as much as running any other.

It stops being sound when they are not. In the setting studied here, a test case is one whole-program execution on one input, and inputs within a single task's pool differ by orders of magnitude in size and in the amount of work they provoke. A deployment pays for seconds, not for executions, so a result reported only in executions does not tell the deployment what it will pay. This is why Chapter 5 makes CPU seconds the primary unit and reports executions alongside rather than instead.

3.2.3 Order Changes the Cost but Not the Answer

Definition 3.2 (Soundness of a schedule)**.** A candidate is accepted only after surviving *every* usable test case in P. Acceptance is therefore independent of the order σ_t: no ordering can accept a patch that another ordering rejects, and no ordering can reject a patch that another ordering accepts.

Definition 3.2 is what makes a pure cost claim possible. If two schedules repaired different numbers of bugs, the difference would be an implementation bug rather than a finding, so the experiment asserts the equality rather than reporting it (Section 5.4).

3.2.4 Test-Case Prioritisation

The problem of choosing σ_t is an instance of *test-case prioritisation*, studied for decades in regression testing. Rothermel et al. [20] formalised it as the search for a permutation of the test pool that maximises how quickly faults are detected, and introduced the two greedy families still in use:

- **Total** strategies rank a test case by a static property, usually how many lines of the program it exercises.
- **Additional** strategies rank by *marginal* gain: how many lines it exercises that nothing scheduled so far has exercised, re-ranking after every pick. This family, written RTP_{add}, is the stronger of the two in most empirical comparisons and is used in this report as a classical baseline.

Elbaum et al. [19] observed that the standard metric for prioritisation assumes uniform test costs, and proposed a cost-cognizant replacement. That paper is the ancestor of the cost term in this work. What it did not have is an *online* cost estimate: its costs are supplied before the run, whereas a repair loop can measure them as it goes, for free, because it is running the tests anyway.

3.2.5 What Frtp Records and What It Discards

FRTP [2] is the direct baseline for the scheduling half of this work. It ranks each test case by its *failing count*: the number of candidate patches that test case has already rejected during this repair session. The information is free, because the repair loop produces it as a by-product, which is why FRTP's overhead is negligible.

Writing $k(x)$ for that count, FRTP schedules in decreasing k. Two things are missing from k, and naming them precisely is what produces this work's scheduling rule.

Figure 3.1 makes both gaps concrete on four tests.

The denominator is missing. Let $n(x)$ be the number of times test case x has been *run*. A count $k(x)$ without $n(x)$ cannot distinguish a test case that was run ten times and rejected nothing from a test case that was never run at all: both have $k = 0$. The ratio $k(x)/n(x)$ is the first quantity that reads the evidence of *not* rejecting, and it is also the first that lets a test case whose early record was luck decay back towards the average.

test	runs (k filled)	k	n	K	cost		
T_1		8	10	0.80		0.05 s	*same k, so a count cannot order them — yet T_2 rejects far less often* and *costs 30× more*
T_2		8	30	0.27		1.50 s	
T_3		0	20	0.00		0.05 s	*same k again — but T_3 has 20 runs of evidence and T_4 has none*
T_4	*never run*	0	0	—	*unknown*		

ranked by count k (Frtp)	$\{T_1, T_2\}$ tied, then $\{T_3, T_4\}$ tied	*two ties, both hiding something*
ranked by rate K	$T_1 \succ T_2 \succ T_4 \succ T_3$	*ties broken; T_3 now ranks last*
ranked by rate ÷ cost	$T_1\,(16.0) \gg T_2\,(0.18)$	*T_2 demoted by 90×*

Figure 3.1: The two things a rejection count leaves out, on four tests. T_1 and T_2 have rejected the same number of patches, so Frtp cannot order them — yet T_1 rejects four candidates in five and T_2 roughly one in four, and T_2 costs thirty times as much to run. T_3 and T_4 also tie at zero, although T_3 has twenty runs of evidence that it never rejects and T_4 has none. Reading the denominator breaks both ties; dividing by cost then demotes the expensive test.

The cost is missing. Nothing in k knows that one test runs in milliseconds and another takes seconds. Under a uniform-cost assumption that is harmless. When costs are spread out it is not, and it is worse than neutral if expensive tests also tend to be more discriminating — in that case ranking on rejections alone actively steers the schedule towards the expensive end of the pool.

3.3 Repair Hypotheses and History in the Prompt

3.3.1 Patches Versus Hypotheses

A *repair hypothesis* is the idea behind a patch: what the model believes is wrong, and how it intends to fix it — "the loop bound is off by one", "the sort key should be the second field", "integer division should be float division". One hypothesis can be written as many different patches, differing in variable names, in statement order, or in whitespace.

This distinction matters for measurement. A *duplicate* patch, in the sense used by Chen et al. [7], is a candidate whose normalised text has already been generated in this session. Two candidates that are textually different but express the same hypothesis are not duplicates by that definition, yet they are just as wasteful: both must be validated, and both fail for the same reason. The duplicate rate is therefore a *lower bound* on the redundancy a repair loop actually produces, which is the sense in which the reported 65 % for ChatRepair is a conservative figure.

3.3.2 Why Showing the History Is Not Enough

The natural fix for redundancy is to show the model what it has already tried. Three published observations say this is necessary but not sufficient.

1. **A transcript does not prevent repetition.** ChatRepair [6] keeps earlier incorrect patches and their failures in the conversation, and Chen et al. [7] still measured 65 % of its generated patches to be duplicates.

2. **Refinement preserves a bad hypothesis.** When the underlying hypothesis is wrong, feedback-driven refinement tends to produce superficial variants of the same idea rather than a new idea [8]. The loop then pays for a new validation that was predictable in advance.

3. **More context can hurt.** Parasaram et al. [18] found that repair effectiveness is *non-monotonic* in the number of facts placed in the prompt: past a point, adding more true and relevant information makes repair worse. A transcript grows monotonically, so it necessarily walks past that point in a long session.

Taken together these say the problem is not the *absence* of history but its *form*. A growing transcript is the worst of both: it costs tokens on every round, it dilutes the few facts that matter, and it still leaves the model free to re-propose.

3.3.3 What a Hint Has to Be

Two properties distinguish a useful hint from a transcript line.

It must be an observation, not a category. A failure class — "the off-by-one family" — can only be phrased as a prohibition: *do not do that again*. A concrete test case can be phrased as a positive requirement: *on this input, produce exactly this output*. A requirement tells the model what success looks like; a prohibition only removes one option out of many and leaves the model to guess the rest. This is also why the scheduler and the hint contract are one system in Caster: the scheduler's unit of memory is the test case, and the test case is precisely the object that can be stated as a requirement.

It must be selected, not accumulated. Because effectiveness is non-monotonic in prompt content [18], the right number of hints is small and roughly constant, not growing. Which hints those are changes from round to round, because the history changes. Selection must therefore happen at every iteration, which is what separates this approach from selecting static facts once before generation begins.

3.4 Benchmark Background

ConDefects [1] collects real faulty submissions from AtCoder programming contests, together with the author's later accepted version and the contest's own test data: 1,254 Java and 1,625 Python faulty programs. It was built to address data leakage in LLM evaluation, since its faults post-date the training cut-offs of the models commonly benchmarked on Defects4J [23].

Four of its properties matter here. Every test input ships with its expected output, so a rejection is a concrete input/output pair rather than a pass/fail bit — which is what lets a hint be stated as a positive requirement. A test is one whole-program execution, so there is no call graph and nothing to *select away*; prioritisation is the only scheduling lever, which is the structural reason the "selection beats prioritisation" finding of Lou et al. [22] does not transfer here. Contest setters ship small public samples alongside large adversarial inputs, so a single task's pool spans a wide range of execution costs — the spread a cost-aware schedule needs. And a submission carries no problem statement, so every prompt must supply worked examples, identically in every configuration, or the comparison measures prompt content rather than policy.

Chapter 4

Methodology

4.1 Overview

CASTER is an iterative repair loop with one extra component: a small record kept for each test case, updated every time that test case is run. Everything else follows from what is in that record.

The record holds three numbers per test case: how many times the test has been run in this session, how many of those runs rejected a candidate, and how much CPU time those runs consumed. From those three numbers the loop derives two estimates — how likely the test is to reject the next candidate, and how long it takes — and spends them in two places. The **scheduler** uses both to decide which test to run first. The **hint selector** uses the first, together with how readable the test's input is, to decide which facts go into the next prompt.

Figure 4.1 shows one round, and Table 4.1 gives the same path as a table of inputs and outputs.

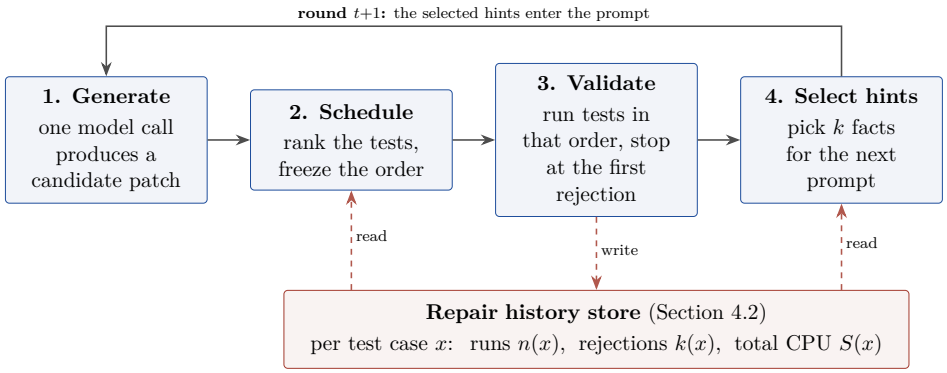


Figure 4.1: One round of the CASTER loop. The repair history store is written by validation and read by both the scheduler and the hint selector, which is why the two mechanisms are one system rather than two independent features.

Table 4.1: Per-round data flow.

Stage	Input	What happens	Output
1	prompt, draw seed	one model call; parse out a candidate program	candidate s_t
2	store (n, k, S)	score every test by Eq. (4.2); freeze the resulting order	order σ_t
3	s_t, σ_t	run s_t on tests in that order; stop at the first rejection	rejecting test $x^\star$, r_t runs, CPU spent
4	$x^\star$, store	select k proven tests; render them as requirements	prompt for round $t{+}1$

4.1.1 Requirements the Design Must Satisfy

Five rules are binding on the implementation, because breaking any of them produces a plausible-looking but wrong number rather than a visible error.

R1 The store holds (n, k, S) per test case, and the test order is derived from it once per round.

R2 The order is *frozen* for the duration of a round: no test is re-ranked once the scan has begun. Re-ranking mid-round would make the order depend on results from the same scan, which is not reproducible from the store alone.

R3 The scan stops at the first rejection. Only tests actually run update n and S; only the rejecting test updates k.

R4 A candidate is accepted only after surviving every usable test, so acceptance is independent of the order (Definition 3.2).

R5 A hint is rendered as a positive requirement over a concrete input/output pair, never as a named prohibition, and the prompt never names a test whose content it does not show.

Two measurement rules accompany them. Cost is read as a `getrusage (RUSAGE_CHILDREN)` delta — CPU time, not wall-clock time — because wall clock on a shared machine is contaminated by sibling processes and would make the learned cost estimate a function of machine load rather than of the test. And because `getrusage` accounting is per process, parallelism across configurations must be process-per-worker rather than threaded, or one worker's CPU is attributed to another.

4.1.2 What the Mechanism Costs

A method that reduces one cost is only worth having if it does not quietly introduce another. It is therefore worth saying precisely what CASTER adds, because the answer is close to nothing: **three numbers per test case, updated in constant time, with nothing trained and nothing executed that the loop was not already going to execute.**

Table 4.2: The overhead CASTER adds to a repair loop, per task. $|P|$ is the number of tests in the pool and B the round budget.

Resource	Added cost	Note
Memory	$O(\|P\|)$	Three numbers per test: n, k and S. A pool of 40 tests is a few hundred bytes
Time per test run	$O(1)$	Two increments and one addition
Time per round	$O(\|P\| \log \|P\|)$	One sort. Built lazily and abandoned at the first rejection, so in practice far less
Extra program executions	**none**	The scheduler reorders the runs the loop was already making; it never adds one
Extra model calls	**none**	Hint selection reads the store. The prompt module cannot execute anything — it does not import the validator
Prompt growth over a session	**none**	At most k hints in any round, not $O(B)$ transcript lines
Training	**none**	There is no model to train and no corpus to collect. The statistics are of this task, in this session
Offline profiling pass	**none**	Test costs are measured from the runs the loop pays for anyway

Two consequences follow. **The information is free**: the scheduler's inputs — which test rejected which candidate, and how long that took — are by-products of work the loop performs regardless, which is the property that made FRTP [2] cheap enough to adopt. And **the component is small and portable**: the store is statistics over test-case identifiers, with no dependency beyond the standard library, no parsing of the program and no knowledge of the language, which is why it unit-tests without a benchmark, a sandbox or a model, and why any loop that can run a test and time it can keep this record.

One caveat, so the claim is not overstated. The *experiment* in Chapter 5 does run an extra full-scan `audit` pass over every patch and every test. That pass exists so the kill matrix and the cost table are established independently of which configuration ran first, which is a requirement of *comparing* schedules fairly. It is not part of the method, and a deployment running CASTER does not perform it.

4.2 The Repair History Store

4.2.1 What Is Stored

For every test case x in the pool, the store holds three counters:

$n(x)$	how many times x has been run in this session
$k(x)$	how many of those runs rejected the candidate
$S(x)$	the total CPU seconds those runs consumed

Nothing else is stored, and in particular nothing about the program under test. The store is therefore pure statistics over test-case names: it can be built, inspected and unit-tested without a benchmark, a sandbox or a model, which is what makes the scheduler testable in isolation.

4.2.2 The Two Derived Estimates

The rejection rate. Treat "does x reject the next candidate?" as a coin flip with unknown bias, and estimate that bias from $k(x)$ successes in $n(x)$ trials. Using the standard Beta conjugate prior, the posterior mean is

$$K(x) \;=\; \frac{k(x)+\alpha}{n(x)+\alpha+\beta}. \tag{4.1}$$

The prior matters more than it looks, so **both choices ship** and the prior is part of the configuration key rather than a constant.

Jeffreys' invariant prior Beta$(\frac{1}{2},\frac{1}{2})$ [24] is the standard non-informative choice and is available as `-prior jeffreys`. It is *not* the default, for a concrete reason: it places a test that has never been run at $K = 0.5$, which outranks a test with a real record of $k = 2$ out of $n = 10$, whose estimate is 0.227. A policy that always prefers the unmeasured test to the proven one degenerates, under uniform costs, into running the whole pool — so under Jeffreys the scheduler stops scheduling.

The default is therefore Beta$(\frac{1}{2}, 2)$, written `-prior shipped`, whose mean is 0.2: above the measured base rate on this corpus and comfortably below one half. Trying new tests is then the job of the exploration term below, not of a side effect of the prior, and the two are not interchangeable. Which prior is better is itself replayable at no cost from the full-scan kill matrix, so both are reported.

The cost estimate. The running mean cost of a test is $\hat{m}(x) = S(x)/n(x)$. A test that has never been run has no mean, and the cold-start value chosen for it decides

how aggressively the loop tries new tests. Entering it at zero would make an unrun test infinitely attractive; entering it at the pool average would make the first round's order essentially arbitrary. CASTER enters an unrun test at the *maximum* of the observed per-test means, which is pessimistic: an unmeasured test must earn its place through its rejection rate rather than through an optimistic guess about its cost. Two alternative cold starts, the median and the 75th percentile, are kept as sensitivity configurations (Section 5.3).

4.2.3 Update Rules

After each scan, the store is updated as follows. Every test that was actually run has its n incremented and its measured CPU added to S. The one test that rejected the candidate, if any, has its k incremented. Tests in the unrun tail of the order are untouched, because they cost nothing and revealed nothing.

One accounting rule is worth stating, because violating it silently corrupts the cost estimate. A test that *times out* is excluded from the cost model rather than entered at the timeout value. A timeout is weaker evidence than a wrong answer — a correct but slow program times out too — and entering timeouts at the clock value would let a handful of them dominate $\hat{m}$ for the whole pool.

4.3 Cost-Aware Validation Scheduling

4.3.1 The Ranking Rule

The scheduler ranks tests by

$$\text{score}(x) \;=\; \frac{K(x)}{\hat{m}(x)^{\,c}}, \qquad c \geq 0, \tag{4.2}$$

and runs them in decreasing score until one of them rejects the candidate.

The intuition is a *rate of return*. $K(x)$ is roughly the probability that spending $\hat{m}(x)$ seconds ends the round; dividing one by the other gives expected rejections per CPU second, and greedily taking the best return first is the natural heuristic for the cost in Equation (3.1). Informally: run the test that is most likely to say "no" per second spent, not the test that is most likely to say "no" overall.

The exponent c interpolates between two positions that already exist in the literature. At $c = 0$ the rule reduces to ranking on the rejection rate alone, which is the FRTP family's position with a denominator added. At $c = 1$ it is the plain ratio $K/\hat{m}$, full cost-cognizance in the sense of Elbaum et al. [19]. Intermediate values discount

the cost term. Because $\hat{m}$ is a noisy online estimate built from few samples early in a session, dividing by the full cost can over-correct: one unusually cheap early measurement can pin a test at the top of the ranking. Values of c below one are the standard way to use a noisy denominator, and c is therefore treated as a declared parameter measured by a sweep (Section 5.3) rather than as a constant fixed at 1 by the name of the method.

Exploration. With probability $\varepsilon = 0.15$, a position in the order is filled uniformly at random from the remaining tests instead of greedily. This is in the report rather than left as an implementation detail because it is load-bearing in two places. Without it, a task whose first test rejects every candidate reaches the end of the session having measured exactly one test: the cost estimate never fills in, so the scheduler has nothing to rank by, and the hint selector has one fact to choose from, which makes the hint-count sweep of Chapter 5 compare k=1 against k=1. And because ε changes measured numbers, it belongs in the configuration key, which means it has to be declared. A configuration with $\varepsilon = 0$ is run as a sensitivity check.

4.3.2 Correcting a Marginal Rate

Equation (4.2) ranks each test on its own record. That is a *marginal* rate, and under stop-at-first-rejection the quantity that decides cost is a *conditional* one.

Consider two tests that reject exactly the same candidates. Each has the same record, so each gets the same K, and both sit near the top of the order. Suppose the first of them runs and the candidate *passes* it. The second test is now nearly worthless: everything it is known to reject has already been shown not to apply here. The marginal K cannot see this, because it never looks at *which* candidates a test rejected, only how many.

This is the same distinction that separates the *total* and *additional* families in test prioritisation [**?**]. The additional family picks the test that adds the most to what previous tests already covered, rather than the test with the best standalone score, and it is the reason the additional variants outperform the total ones in that literature. The rule here is the same idea with a different notion of coverage: not lines of code, but the set of candidates a test is known to reject.

Write $R(x)$ for the set of candidates test x has been observed to reject so far in the session, and let P be the tests that have already run *and passed* on the current candidate. Define

$$\text{overlap}(x \mid a) \;=\; \frac{|R(x) \cap R(a)|}{|R(x)|}, \tag{4.3}$$

and defer x — move it to the back of this round's order — when $\text{overlap}(x \mid a) \geq \tau$ for some $a \in P$. The default is $\tau = 0.75$.

Three properties of this rule matter more than the rule itself.

It defers, it never drops. A deferred test still runs; it runs last. An accepting round therefore still executes every test, so Definition 3.2 is untouched and scheduling remains a pure cost claim. A rule that skipped tests would be a different and unsound method.

A test is deferred at most once per round. Without that bound a test can be stepped over repeatedly as P grows, which pushes a genuine rejecter arbitrarily far back. The bound is also the better rule when measured: on the pilot data, a deferral lasting the whole round is worth $-4.4\,\%$ CPU with a $95\,\%$ interval of $[-9.1, +0.2]$, against $-6.3\,\%$ with $[-13.5, -0.6]$ for one deferral per test. Bounding the correction makes it both safer and stronger.

The rule is cheap; the evidence is not. Equation (4.3) needs two tests to have been seen rejecting the same candidate. Under stop-at-first-rejection a round records exactly *one* rejecting test, so that only happens when a candidate recurs in a later round and a different test reaches it first — which occurred in $33\,\%$ of pilot sessions, usually for a single candidate. Measured on session-local evidence alone the correction is worth $-0.12\,\%$ CPU, interval $[-1.6, +1.3]$, which is not distinguishable from zero. Seeded instead from the rejection sets the audit pass has already computed for the same task, the same rule is worth $-6.3\,\%$, interval $[-13.5, -0.6]$. The configuration with $\tau = 0$ is run as a separate arm so that this clause is priced on its own rather than assumed.

The order costs nothing to abandon. It is built one position at a time from the store as it stood when the round began, and construction stops at the first rejection, so the unprobed tail is never materialised. Scheduling also cannot change *which* candidates are accepted: by Definition 3.2 a candidate is accepted only after surviving every usable test, so every schedule accepts exactly the same ones. The scheduling claim is a pure cost claim, and a difference in repair rate between two schedules is a soundness bug rather than a result.

4.4 Selected Hints

4.4.1 What a Hint Is

A hint is one test case, shown to the model as a requirement the next patch must satisfy:

```
On this input:
  5 3
  1 4 2 8 5
your program must print exactly:
  14
(kills 43% of attempts)
```

Three properties of this form matter, and each is a rule in the implementation.

It is a requirement, not a prohibition. The hint says what the patch must do, not what it must avoid. A prohibition ("stop using integer division") removes one option and leaves the rest of the space unconstrained; worse, naming a failure mode in a prompt can make the model more likely to produce it. A requirement constrains the output directly and can be checked by the model as it writes.

Only proven tests are shown. A test whose estimate comes entirely from the prior, with no actual record, is a guess, and a wrong guess spends tokens to point the model in a direction nothing supports. A test must have rejected at least one candidate in this session before it can appear as a hint.

The number shown is the posterior rate, and it is worth being explicit about where it comes from. The figure in the last line is $K(x)$ of Equation (4.1) — the same Beta-posterior rejection rate the scheduler ranks by, rounded to the nearest percent. It is *not* the ranking score $K/\hat{m}^c$, which mixes in cost and would be meaningless to the model, and it is *not* the raw frequency $k(x)/n(x)$.

Under the default prior $\mathrm{Beta}(\frac{1}{2}, 2)$ the shown value is

$$K(x) \;=\; \frac{k(x) + 0.5}{n(x) + 2.5}, \tag{4.4}$$

so the 43 % in the example above is a test that has rejected $k = 4$ of the $n = 8$ candidates it has been run against: $(4 + 0.5)/(8 + 2.5) = 0.4286$.

Using the posterior rather than the raw frequency is deliberate. A test with $k = 1$ out of $n = 1$ has a raw rate of 100 %, and telling the model that every attempt fails this check is a claim no single observation supports; the posterior reports $1.5/3.5 = 43\,\%$ instead. The cost of that choice is visible in the same two numbers: $k{=}1, n{=}1$ and $k{=}4, n{=}8$ both display as 43 %, because at small n the prior is doing most of the work. The figure is therefore an estimate of how likely the next candidate is to fail this test, not a count of how often past candidates did, and the sentence that carries it is worded as an estimate rather than as a tally.

4.4.2 How Hints Are Selected

At every round, the selector ranks proven tests by

$$\text{hintscore}(x) \;=\; K(x) \times \text{legibility}(x) \tag{4.5}$$

and takes the top k, with $k = 3$ by default — but it does not take them blindly, and the two qualifications are the point of this subsection.

Near-duplicate hints are dropped. Two hints that reject the same candidates spend two slots in the prompt to say one thing. The model gets no second failure mode and no second constraint, only the first one restated in different bytes, and it pays tokens and attention for the restatement. The selector therefore walks the ranking greedily and skips a candidate hint whose rejection set is already covered by one it has chosen, using the same containment of Equation (4.3) and the same threshold τ:

$$\max\Big(\text{overlap}(x \mid m),\ \text{overlap}(m \mid x)\Big) \;\geq\; \tau \quad \text{for some already-chosen } m \implies \text{skip } x. \tag{4.6}$$

The comparison is symmetric here, unlike in the scheduler. There, one test has already run and the question is whether the next one still has anything to add after it; here neither hint has run and the question is whether the *pair* is worth two slots, so a narrow hint sitting inside a broad one is dropped whichever way round the two are ranked.

k is a ceiling, not a count. Raising the default from 2 to 3 is what the deduplication is for: asking for three distinct hints renders about what asking for two undeduplicated hints rendered before, with the third slot going to a genuinely different failure mode when one exists. Often one does not. In the pilot, 55 % of sessions ended with fewer than two distinct proven rejecters at all, so the selector returns fewer than k far more often than it returns k. Every record therefore carries the number of hints that actually reached the prompt alongside the number requested, and the hint-count sweep

of Chapter 5 is read against the former. A sweep read against the requested k would report a comparison it did not run.

The cost term $\hat{m}(x)$ is deliberately *absent* from Equation (4.5). The scheduler and the selector spend different currencies: a scan pays CPU seconds, a prompt pays tokens. A test that takes five seconds to run is expensive for the scheduler and free for the prompt, so carrying the cost term into hint selection would discard the most informative facts for no benefit.

The legibility term is what replaces it. An input of a few bytes can be shown whole; an input of several megabytes cannot, and pasting a truncated prefix of it produces a requirement the model cannot satisfy — it would be told to produce a specific output on an input it has not seen. Inputs above a length threshold of 400 characters are therefore rendered as a *shape*, the first line verbatim followed by the structure and size, and their score is multiplied by a penalty of 0.45.

That the penalty is a factor rather than a veto is deliberate: a test that rejects nearly everything is still the most useful fact available, so a dominant illegible test can outrank a legible one when nothing else is close. The identifier of a test case is never shown at all, because a file name carries no information the model can act on and invites it to reason about the benchmark rather than the program.

4.4.3 The Disclosure Dial

How much of a test case the prompt reveals is a parameter, not a fixed choice, because it trades two risks against each other. Showing less gives the model too little to work with. Showing more invites it to satisfy *the shown tests* specifically — writing a special case for the displayed input rather than repairing the fault — which produces a patch that passes the hinted tests and fails everything else.

Caster therefore exposes three levels, measured as a dose-response curve in Chapter 6:

Table 4.3: Disclosure levels, from least to most revealed.

Level	What the prompt shows
`sample-only`	**Nothing from the store.** The contract block is withheld in full, so the only worked examples in the prompt are the task's own public samples. This is the floor of the ladder, not a partial disclosure.
`no-required-line`	The selected *inputs*, and a line saying the patch is checked on them — but their expected outputs are withheld.
`full` (default)	The selected inputs, their expected outputs, and an explicit statement that the patch must produce them.

The number of hints k and the disclosure level are the same dial seen from two sides, and Chapter 6 reports them as one curve rather than two findings.

4.4.4 Why This Is Not a Transcript

Three differences from transcript memory follow from the construction.

The prompt does not grow. At most k hints appear in any round, and the bound does not depend on how long the session has run. It is *at most* rather than exactly: a round reaches the prompt with fewer when the store holds fewer than k proven tests, and with none at all in round 1, when the store is empty — which is also what makes round 1 byte-identical across every configuration. A transcript, by contrast, grows linearly in the number of rounds and therefore walks past the point where more context starts to hurt [18].

The content changes. Which tests have the highest rejection rate changes as the session proceeds, so round 10 can carry entirely different hints from round 3. A transcript is append-only: round 10 contains everything round 3 contained.

The facts are earned, not asserted. Each hint is backed by a measured record in the store, produced by the scheduler that had to run the test anyway. The hint mechanism adds no executions of its own — it reads the store, it never runs anything — which is why the two mechanisms compose without their costs compounding.

Chapter 5

Experiments

This chapter states what is being asked, on what data, against what, and how the answers will be judged. It is written before the results so that every quantity in Chapter 6 has a declared meaning and a declared success criterion rather than one chosen after the numbers arrived.

5.1 Research Questions

Four questions. Each is paired with the outcome that would *refute* it, because a question with no refuting outcome is a name for a group of experiments rather than a question.

RQ1 — Does cost-aware validation scheduling reduce the cost of rejecting a patch? Does ranking tests by $K/\hat{m}^c$ — an online rejection rate over an online cost — reduce the CPU spent on validation, compared with the order the benchmark ships, an uninformed random order, the published repair-loop prioritiser FRTP [2], and the classical additional-greedy coverage prioritiser [20]?

Refuted when the scheduler is not cheaper than the shipped order beyond the bootstrap interval; or when it is cheaper than the shipped order but not cheaper than FRTP and coverage, in which case the contribution is early stopping, which is not ours.

RQ2 — Do selected hints reduce redundant generation? Does handing the model a small, freshly selected set of concrete requirements move its proposals off the ideas it has already tried — fewer duplicate patches, and at least as many repairs at the same budget — where independent repeated sampling, a prohibition, and a full conversation transcript do not?

Refuted when the duplicate rate does not fall against independent repeated sampling; or when it falls and the repair rate does not rise, in which case the distribution moved somewhere no better.

RQ3 — Do the two mechanisms reinforce each other? The two mechanisms are claimed to compose: scheduling produces the failure record that hints are selected from, and better hints send fewer wasted candidates to the expensive validation step. Does the combined system beat the sum of what each does alone, on total cost and on repairs within budget?

Refuted when the combination is no better than the better of the two alone, in which case the mechanisms are independent and should be reported as two separate contributions rather than one system.

RQ4 — Which parts carry the effect, and how sensitive are they? Three things separate the proposed scheduler from the shipped order: having a memory at all, turning a count of rejections into a rate, and dividing that rate by cost. Which one produces the effect? And does the answer survive the design choices — the cost exponent c, the cold-start rule, the exploration rate, and the number of hints shown?

Refuted when no single step clears the bootstrap interval, so the gain is simply "having any memory" and the score function is ornament; or when the answer flips under a routine change of one parameter, in which case the result is an artefact of tuning.

5.2 Benchmark: ConDefects Python

5.2.1 Corpus, and the 100-Task Pilot Drawn From It

The corpus is the Python subset of ConDefects [1], chosen for the four reasons given in Section 3.4: every test input ships with its expected output, a test is one whole-program execution, the inputs within a single task are deliberately heterogeneous in cost, and the faults post-date the training cut-offs of commonly benchmarked models.

The evaluation reported here is a **pilot of** 100 **tasks** drawn from that corpus; the scale and its reasons are in Section 5.2.2 below, and running the full corpus is the first item of future work (Section 9.2).

A task is admitted to the corpus when every one of the following holds. The screen is applied before any configuration runs, and the admitted set is frozen with a checksum so that no later analysis can change it.

To be filled from the 100-task pilot. The admission funnel — how many faults each criterion removed, and how many tasks the 100 were drawn from — is reported in Section 6.1.

Table 5.1: Corpus admission criteria. Applied once, before treatment.

Criterion	Why
Single-file Python fault	The scope of this study; matches the sandbox model of one program, one input.
Faulty and reference versions both present	The reference supplies the expected outputs the oracle compares against.
At least one test rejects the faulty version	Otherwise the task is already "repaired" at round 0 and measures nothing.
The reference passes every test in the pool	A test the correct program fails is an instrument fault, not a fault in the candidate; such tests leave the pool.
The reference finishes within the sandbox clock on every pooled test	Applying the clock to the reference first is what keeps the oracle sound: a test the correct program cannot finish is not a test at all.
Pool size at least the declared minimum	A scheduling experiment on a pool of three tests measures nothing about scheduling.

5.2.2 Pilot Scale and Why It Is a Pilot

Table 5.2: Pilot configuration.

Parameter	Value	Note
Tasks	100	Drawn from the admitted ConDefects Python corpus
Seeds per task	3	Different draws from the same model, same task
Budget B	20 rounds	Maximum candidate patches per session
Runs per configuration	6,000 rounds	$100 \times 3 \times 20$ at the budget ceiling
Sandbox clock	10 s wall	A hard kill, applied to the reference first
Proposer model	`qwen2.5-coder:7b`	One open-weights model served locally through an OpenAI-compatible endpoint, held fixed across every configuration
Temperature	1.0	Fixed; the study varies policy, not decoding

This is a pilot and is reported as one. Its purpose is to establish whether the effects exist and are large enough to be worth a corpus-scale run, and to fix the parameters that the full run must declare in advance. The full ConDefects Python corpus is roughly an order of magnitude larger, and running it is the next step (Section 9.2).

5.2.3 Screens Are Bands, Decided After the Run

One protocol decision governs the whole corpus and is worth stating before the configurations, because it determines what can be asked afterwards.

A *screen* rejects a task before any measurement exists; a *band* keeps every task and groups them at analysis time. They are not equally reversible — a screen must be decided before the first cell exists and cannot be undone, whereas a band can be re-cut any number of ways from the same rows — so where both are available this study takes the reversible one. The corpus admits every task whose oracle is sound: the criteria in Table 5.1 are only those without which a measured number would be *wrong*, never those that merely shape the population.

Table 5.3 declares the three band axes before the corpus run, one per condition of Section 7.1.

Two rules keep the difficulty axis honest, because it is the one that could be abused. The rate is taken from a single designated floor configuration (`no-mem`), not from the configuration under test, so no policy can band itself into a favourable population. And it is frozen once, beside the corpus checksum, and never recomputed by an analysis script — a rate recomputed on every call is a rate that moves when the merge moves, and every banded number downstream would move with it without saying so.

Table 5.3: The three band axes, their boundaries, and when each becomes known. Boundaries are fixed here so that the banded result in Chapter 6 is not a cut chosen after seeing which one looks best.

Axis	Bands	Status
Pool size $\|P\|$	`degenerate` $= 1$; `tiny` 2–9; `small` 10–19; `standard` ≥ 20	A property of the frozen pool, known before any candidate is drawn. A *stratification*
Within-task cost spread	`flat` / `mixed` / `spread`, by terciles of the within-task coefficient of variation of per-test CPU	A property of the frozen cost table, established by the arm-independent `audit` pass before any configuration runs. A *stratification*
Task difficulty	`dead` $= 0$; `hard` < 0.25; `mid` 0.25–0.60; `easy` 0.60–1; `trivial` $= 1$	The repair rate of the floor configuration, per model. An **outcome**, so it may only ever be a post-hoc band and never an admission criterion

The `standard` pool band is exactly what a pre-run screen of $|P| \geq 20$ would have admitted, so every number such a screen would have produced is still computable, and the tasks it would have deleted are reported beside it rather than discarded.

5.3 Baselines and Configurations

5.3.1 The Two Blocks

Configurations fall into two blocks that differ in what they cost to run, and the difference governs the order of work.

Block A varies only the *order* in which tests are run. The prompt is identical across these configurations, so the candidate patches can be drawn once and reused: adding a Block A configuration costs CPU but no model calls.

Block B varies the *prompt*. Here the round-t prompt depends on what validation found in rounds $1..t-1$, so the draw and the schedule cannot be separated and each configuration needs its own set of model calls.

One identity ties the two blocks together. In round 1 the store is empty, so a steered prompt carries no hints and is byte-identical to an unsteered one. Round 1 must therefore be the same draw in every configuration, which requires the draw seed to be a function of the task, the seed and the round number only — never of the policy. Deriving it from the full configuration key would make every configuration buy its own round 1 and would silently destroy the pairing that every comparison in Section 5.7 depends on.

5.3.2 Block A — Scheduling Configurations

Table 5.4: Block A: configurations that differ only in test order. "Answers" names the research question each one serves.

Name	Order of tests	Answers
`full-suite`	Every test, no early stop	calibration
`no-mem` *(independent repeated sampling)*	Shipped order, and the store is never updated, so every round redraws from scratch. The name is the implementation's; what it *is* is independent repeated prompting	calibration, RQ2 baseline
`natural`	The order the benchmark ships — **the incumbent**	RQ1
`random`	Seeded shuffle, reshuffled each round — **the control**	RQ1
`frtp`	Proven rejecters by *count* descending, then the rest [2]	RQ1, RQ4
`coverage`	Additional-greedy over reference line coverage [20]	RQ1
`cost-only`	$\hat{m}$ ascending; no rejection information	RQ4
`kill-only`	K descending; no cost information	RQ4
`sched`	$K/\hat{m}^c$, plus the overlap deferral of Section 4.3.2 — **the proposed scheduler**	RQ1, RQ3
`sched-ov0`	The same, with $\tau = 0$: the overlap deferral switched off, so the pair prices that clause and nothing else	RQ4
`sched-cs-median`	Cold start at the median or the 75th percentile instead of the max	sensitivity
`sched-cs-q75`		
`sched-bucket`	Discrete buckets instead of a continuous score	sensitivity
`sched-explore0`	$\varepsilon = 0$	sensitivity, RQ4 precondition

Two framing points govern how this table is read.

`natural` is the incumbent, not a neutral control. Sorting by file name is not a scheduling decision anyone made; it is what any implementation gets for free, and therefore what most implementations do. It is also not arbitrary: contest naming puts the small, public, highly discriminating sample cases first, so the shipped order is *better* than chance. The uninformed control is `random`, and reporting both is what separates "better than nothing" from "better than the thing people actually run".

`frtp` is the published baseline, implemented as published. It ranks on the rejection *count*, which is what FRTP ranks on, not on a rate and not on anything divided

by cost. A baseline weakened to make the proposal look better answers nothing.

5.3.3 Block B — Prompt Configurations

Table 5.5: Block B: configurations that differ in what the prompt carries. All use the proposed scheduler unless the name says otherwise.

Name	What the prompt carries after round 1	Answers
`hints`	k=3 selected requirements, **shipped test order**	RQ2, RQ3
`sched+hints`	k=3 selected requirements **and** the proposed scheduler	RQ2, RQ3
`exclusion`	The same facts phrased as a prohibition	RQ2
`chat`	The full conversation transcript, ChatRepair-style [6]	RQ2
`hints-k1`, `hints-k2`	$k \in \{1, 2\}$ instead of 3. Read against the number of hints *rendered*, not requested: deduplication and a thin pool mean the selector returns fewer than k in most sessions	RQ4
`hints-sample`	Disclosure `sample-only`	RQ4
`hints-noreq`	Disclosure `no-required-line`	RQ4

`no-mem` is the baseline for RQ2, and it is a strong one. It redraws independently every round under a policy-free seed, which is exactly *independent repeated prompting*. Chen et al. [7] measured that strategy at 23 bugs repaired against ChatRepair's 18 at an equal budget of 24 model calls, with 65 % of the conversational system's patches being duplicates. The obvious iterative strategy is therefore already published as *worse* than simply drawing again. Block B must beat `no-mem`; beating `natural` proves nothing on the generation side.

5.3.4 The Factorial for RQ3

RQ3 asks whether the two mechanisms reinforce each other, which requires each of them alone and both together on the same tasks and seeds. Table 5.6 is that design.

Table 5.6: The 2×2 design behind RQ3. Each cell is a configuration run on the same 100 tasks $\times$ 3 seeds.

	Shipped test order	**Cost-aware scheduling**
No hints	`no-mem` (baseline)	`sched`
Selected hints	`hints`	**`sched+hints`** (full system)

5.3.5 Parameter Sweeps

Two parameters are measured rather than assumed. The **cost exponent** c of Equation (4.2) is swept over the range $[0, 1.25]$, whose two endpoints are the two published positions: $c = 0$ is ranking on the rejection rate alone, $c = 1$ is full cost-cognizance. The **hint dose** is swept over $k \in \{1, 2, 3\}$ crossed with the three disclosure levels of Table 4.3.

Both sweeps are reported as sensitivity, not as headline results. A value selected on the same data it is reported on is optimistically biased, so the full corpus run must declare c and k in advance and report the sweep as a check (Section 9.2).

5.4 Metrics

The study does not collapse into a single figure of merit, because the two halves of the loop spend different resources and a composite would let one pay for the other.

Table 5.7: Validation-side metrics (RQ1, RQ3, RQ4).

Metric	Definition	Success criterion
CPU seconds per session	`getrusage` delta summed over the tests actually run; paired per task and seed	**Primary.** The scheduler below `natural`, `random`, `frtp` and `coverage`, beyond the bootstrap interval
Test executions per session	How many tests were run; reported because the prioritisation literature reports executions	Reported for every configuration; *not* a success criterion on its own
CPU per execution	CPU divided by executions; separates "cheaper" from "fewer"	Diagnostic: it is where the mechanism claim lives
Stepwise decomposition	Paired Δ CPU for each one-variable step of the ablation ladder (Table 5.9)	At least one step clears the interval

Hint leakage needs no held-out tests. The leakage metric is computed entirely from data the main grid already produces: each round record stores the facts the prompt showed, and the `audit` full-scan pass stores, for every patch, the complete set of tests that reject it. The share of rejections falling outside the disclosed set is then a set difference over those two, so the measurement works at the default setting where the whole pool is the oracle and nothing is held out.

A *second*, narrower question does need held-out tests: of the patches the loop *accepted*, how many fail on tests the loop never ran? With no holdout, acceptance

Table 5.8: Generation-side metrics (RQ2, RQ3, RQ4).

Metric	Definition	Success criterion
Duplicate rate	1 − distinct/total patches in a session, over normalised patch text	**Primary for RQ2.** Falls against `no-mem`
`success@B`	Share of sessions that reach a plausible patch within budget B	Rises, or at minimum does not fall, wherever the duplicate rate falls
First-accept round	The round at which the first plausible patch appears	Reported paired
Prompt and completion tokens	The generation bill, since the LLM-repair literature prices repair in tokens	Reported for every Block B configuration
Hint leakage	Of the patches that are rejected at all, the share whose rejecting tests are *disjoint* from what the prompt showed — the patch satisfied everything it was told and died elsewhere. Computed from the `facts_shown` field on every round record against the full rejection vector produced by the `audit` pass	**Primary for the RQ4 dose curve.** Should fall monotonically as disclosure rises, or the disclosure axis does not exist

is defined by the same tests that would have to judge it, so no post-hoc analysis can recover that number. That question is answered by a separate pass with `-holdout` on a *subset* of tasks; it is a different question from RQ1 and does not need the full grid, so it does not require re-freezing the pilot corpus.

Guaranteed, not measured. By Definition 3.2, `success@B` and the first-accept round are identical across every Block A configuration, on every task and seed, not merely on average. The experiment therefore *asserts* that equality and halts if it is violated, rather than reporting it as a finding. A difference there would be a soundness bug.

5.5 The Ablation Ladder

An ablation chain must change one thing per step or it cannot attribute the effect. Going directly from `frtp` to `sched` changes two things at once — a count becomes a rate, *and* the rate is divided by cost — so the ladder is built to separate them.

Table 5.9: The ablation ladder for RQ4. One variable per step.

Step	What changes	What it means if this step does not clear the interval
`natural` → `cost-only`	Cost enters; no rejection information	Cost alone is not a usable signal
`natural` → `kill-only`	Rejection rate enters; no cost	The memory term is ornament
`natural` → `frtp`	The published baseline's step	FRTP does not reproduce in this setting
`frtp` → `kill-only`	A **count** becomes a **rate**	Reading the denominator buys nothing
`kill-only` → `sched`	The rate is divided by cost	Cost-awareness is ornament
`sched` → $K/\hat{m}^c$	The exponent is tuned	c need not be a parameter

5.6 Implementation and Measurement

5.6.1 The Tool

CASTER is a Python package of command-line verbs; the ones needed to reproduce the pilot are `freeze` (apply the admission screen and write the frozen corpus), `propose` (fill the response cache and patch store), `audit` (a full-scan pass establishing, for every patch, which tests reject it and what each costs), `evaluate` (walk the task × seed × configuration grid, resumably), then `merge`, `analyse` and `report`. Three module boundaries are asserted by unit tests: only the benchmark adapter reads the benchmark, the store has no file access and no benchmark imports, and the prompt module does not import the validator. The package ships 263 unit tests that run without a network, a model or the benchmark.

5.6.2 How Cost Is Measured

Cost is the delta of `getrusage(RUSAGE_CHILDREN)` across one test run: CPU seconds, never wall clock. Wall clock on a machine running several workers is contaminated by siblings, which would make the learned cost estimate a function of machine load rather than of the test. Because `getrusage` accounting is per process, parallelism is process-per-worker and the sandbox refuses to run under a thread pool rather than let one worker's CPU be attributed to another.

Three further rules keep the measurement honest.

1. **The cost table is established by the audit pass, not by any configuration.** If a configuration were allowed to populate the cost table, that table would become

a function of which configuration ran first, and every later comparison would inherit the bias.

2. **No (patch, test) pair is executed twice.** Outcomes are memoised, and the identity (fresh runs − timeout re-runs) + cache hits = $\sum$ test calls is instrumented per scan and over the whole run rather than asserted in prose. A stored outcome that disagrees with a fresh one halts the run.

3. **Timeouts are quarantined.** A timed-out test is never served from the cache and never enters the cost model, because a correct-but-slow program times out too and entering timeouts at the clock value would let a handful of them dominate the cost estimate.

5.6.3 Reproducibility

The admitted corpus, the pool order and the parameter values are frozen before any configuration runs, under a checksum. Every value that can change a measured quantity is either part of a cache key — model, temperature, prompt, output budget, draw seed — or recorded in the run's protocol record, which carries the sandbox clock, the interpreter version, the backend and the model digest. Merging results across machines refuses to join across a disagreement in that record, so a parameter changed halfway through a run produces a refusal rather than a quietly mixed average.

5.7 Statistical Analysis

Pairing. Every comparison is paired at the level of a (task, seed) cell under common random numbers, so that two configurations see the same round-1 candidate on the same task. Unpaired comparison on 100 tasks with the cost spread this corpus has would be dominated by which tasks landed in which arm.

Intervals. Per-task totals are not independent across seeds, so confidence intervals come from a *cluster bootstrap over tasks* — resampling whole tasks with replacement, 10,000 resamples — rather than from a bootstrap over cells, which would treat three seeds on one task as three independent observations and report an interval that is too narrow.

Tests. Paired differences are tested with the Wilcoxon signed-rank test over per-task totals. The distribution of per-task CPU is heavily right-skewed, so a t-test on the mean would be driven by a few expensive tasks.

Effect size. Differences are reported as a percentage change in total CPU together with the number of tasks on which the comparison goes each way. The second number matters: an aggregate win driven by three expensive tasks is a different claim from a win on eighty tasks, and the per-task scatter in Chapter 6 shows which it is.

Multiple comparisons. Four primary contrasts are declared in advance: `sched` against `natural`, `sched` against `frtp` (RQ1), `sched+hints` against `no-mem` on duplicate rate (RQ2), and `sched+hints` against the better of `sched` and `hints` (RQ3). These four carry a Holm correction. Every other contrast in Chapter 6 — the ablation ladder, both sweeps, and the sensitivity table — is reported with its interval and labelled exploratory, and no claim rests on one of them alone.

Chapter 6

Results

To be filled from the 100-task pilot. The 100-task pilot is running at the time of writing. Every table and figure in this chapter is laid out with its final shape, its units and its reading, and every measured value is marked —. Nothing in this chapter is to be read as a result until the marks are replaced.

6.1 Corpus and Measurement Summary

Before any comparison, two descriptions of the corpus are needed, because they decide whether the question RQ1 asks is even well posed. If test costs within a task were uniform, cost-aware scheduling would have nothing to correct.

Table 6.1: Admission funnel: how the 100 pilot tasks were obtained.

Stage	Faults remaining	Removed
ConDefects Python, all faults	—	—
Faulty and reference versions both usable	—	—
At least one test rejects the faulty version	—	—
Reference passes every pooled test	—	—
Reference finishes within the sandbox clock	—	—
Pool size at or above the minimum	—	—
Admitted	—	
Drawn for the pilot	100	

6.2 RQ1: Effectiveness of Cost-Aware Scheduling

6.2.1 Head-to-Head

The contrast against `natural` establishes that scheduling is worth doing at all; the contrast against `frtp` tests this work's specific claim, because FRTP already has a memory, so any difference there is attributable to the rate and the cost term rather than to remembering.

Table 6.2: The two distributions that decide whether cost-aware scheduling has anything to work with.

Quantity	Value	How to read it
Tasks / tests / patches	—	The size of the pilot universe
Per-test CPU cost. If the high percentiles sit close to the median, costs are near-uniform and the cost term has nothing to correct. The spread *within* a task is the one that matters.		
median	— s	
90th percentile	— s	
99th percentile	— s	
median within-task coefficient of variation	—	
Per-test rejection rate. If a handful of tests reject everything, any memory finds them quickly and every configuration converges to the same behaviour.		
mean	—	
median	—	
tests that never reject	— %	
Median within-task $\mathrm{corr}(\log c, K)$	—	**The sign decides whether dividing by cost is a refinement or a correction. If positive, expensive tests are *more* likely to reject, so ranking on rejection alone is actively drawn towards the expensive end of the pool.**

Table 6.3: RQ1 head-to-head over the pilot (`frtp` [2], `coverage` = $\mathrm{RTP}_{\mathrm{add}}$ [20]). The upper block is absolute cost; the lower block is the paired contrast against `sched`, with cluster-bootstrap intervals over tasks (10,000 resamples) and Holm-corrected Wilcoxon p. **CPU / exec.** separates "cheaper" from "fewer": if `sched` wins on CPU while paying *more* executions, the two units of account disagree in sign.

Configuration	Total CPU (s)	Exec. / round	CPU / round (s)	**CPU / exec. (s)**
`natural` (incumbent)	—	—	—	—
`random` (control)	—	—	—	—
`frtp`	—	—	—	—
`coverage`	—	—	—	—
`sched` (ours)	—	—	—	—
Paired contrasts				
Contrast	Δ CPU	95 % CI	p / tasks won	Δ exec.
`sched` vs `natural`	—	—	—	—
`sched` vs `random`	—	—	—	—
`sched` vs `frtp`	—	—	—	—
`sched` vs `coverage`	—	—	—	—
`frtp` vs `natural`	—	—	—	—

6.2.2 Results by Band

Section 7.1 states three conditions under which the method is expected to pay off, and Section 5.2.3 fixed the band boundaries before the run. Per-band tables for all three axes are produced by `analyse`; the one that carries the argument is Table 6.4, which walks from the whole corpus to the sub-population where all three conditions hold.

Table 6.4: The two headline views. The first row is what a deployment gets pointing the tool at everything; the second is what the mechanism does where the mechanism applies.

Population	tasks	vs `natural`		vs `frtp`	
		Δ CPU	95 % CI	Δ CPU	95 % CI
Whole corpus	100	—	—	—	—
C1 only: $\|P\| \geq 10$	—	—	—	—	—
C1 + C2: and costs not flat	—	—	—	—	—
C1 + C2 + C3: and not `easy/trivial`	—	—	—	—	—

The expected shape is a gradient, and the gradient is the argument: each step down removes a sub-population where no effect is mechanically possible. A flat result across the three steps would mean the conditions of Section 7.1 are not the operative ones, and the explanation of *why* the method works is wrong even if the headline number survives. Two per-band checks are soundness rather than findings: the `degenerate` pool band and the `trivial` difficulty band must both show no difference at all, because in each every configuration runs one identical scan.

6.2.3 Learning Curve

Figure to be generated

Mean validation CPU per round, rounds 1 to 20, one line per configuration. Round 1 is identical across configurations by construction, because the store is empty.

Figure 6.1: Validation CPU per round. Round 1 is the same for every configuration; what happens afterwards is the store filling in. An incumbent line that stays flat is the loop never learning anything about its pool.

To be filled from the 100-task pilot. The reading to check: how many rounds the memory configurations need before they separate from `natural`, and whether `sched` continues to separate from `frtp` after the rejection information has saturated — the cost estimates keep improving after the rejection counts stop changing, so continued separation is the signature of the cost term doing work.

6.2.4 Soundness Check

Table 6.5: Soundness: every Block A configuration must accept exactly the same patches on exactly the same rounds (Definition 3.2). A difference here is an implementation bug, not a finding.

Configuration	`success@B`	First-accept round (mean)
`natural`	—	—
`random`	—	—
`frtp`	—	—
`coverage`	—	—
`sched`	—	—

6.3 RQ2: Effectiveness of Selected Hints

Table 6.6: Generation-side results. The baseline is `no-mem`, which is independent repeated prompting — the strategy Chen et al. [7] found to beat conversational repair; `chat` is a ChatRepair-style transcript [6].

Configuration	Dup. rate	`success@B`	First acc.	Prompt tok.	Compl. tok.
`no-mem` (baseline)	—	—	—	—	—
`chat` (transcript)	—	—	—	—	—
`exclusion` (prohibition)	—	—	—	—	—
`hints` (ours)	—	—	—	—	—

Table 6.7: Paired contrasts for RQ2, on the duplicate rate.

Contrast	Δ duplicate rate	95 % CI	p	tasks won
`hints` vs `no-mem`	—	—	—	—
`hints` vs `chat`	—	—	—	—
`hints` vs `exclusion`	—	—	—	—

Three comparisons run at once and they answer different things. Against `no-mem`, whether showing anything at all helps — a real bar, since independent repeated sampling is published as the stronger of the two obvious strategies. Against `chat`, whether

selecting beats *accumulating*. Against `exclusion`, whether the *form* matters: it carries the same facts phrased as what not to do, so a difference there isolates framing from content. The token columns matter because a transcript grows every round while `hints` stays flat by construction — a duplicate-rate win that costs twice the tokens is a different result.

Figure to be generated

Duplicate rate by round, one line per Block B configuration, rounds 1 to 20. Round 1 carries no hints in any configuration.

Figure 6.2: Duplicate rate over the course of a session. The gap between configurations should widen with round number, because that is when the history exists to be selected from.

6.4 RQ3: Combined Effect

Table 6.8: The 2 × 2 design of Table 5.6, measured. The combined cell is the full system.

Configuration	Total CPU (s)	Exec. / round	Dup. rate	`success@B`
`no-mem` (neither)	—	—	—	—
`sched` (scheduling only)	—	—	—	—
`hints` (hints only)	—	—	—	—
`sched+hints` (both)	—	—	—	—

Table 6.9: Interaction. "Additive prediction" is what the combined cell would cost if the two mechanisms did not interact; the residual is the interaction term.

Quantity	Additive prediction	Measured	Residual (95 % CI)
Total CPU	—	—	—
Executions / round	—	—	—
Duplicate rate	—	—	—
`success@B`	—	—	—

The claimed feeding is specific enough to check: hints cut how many redundant candidates reach validation, which lowers CPU beyond what re-ordering alone can; scheduling supplies the rejection record hint selection ranks on. A residual indistinguishable from zero means the two are independent improvements that happen to ship

together — still useful, but then they are two contributions rather than one system, and RQ3 is refuted.

6.5 RQ4: Ablation and Sensitivity

6.5.1 Which Term Carries the Scheduling Effect

Table 6.10: The ablation ladder of Table 5.9, measured. One variable changes per row. Exploratory: intervals are reported without correction and no claim rests on a single row.

Step	What changes	Δ CPU	95 % CI	Conclusion if it does not clear
`natural` → `cost-only`	cost only	—	—	Cost alone is not a usable signal
`natural` → `kill-only`	rejection only	—	—	The memory term is ornament
`natural` → `frtp`	published step	—	—	FRTP does not reproduce here
`frtp` → `kill-only`	count → rate	—	—	The denominator buys nothing
`kill-only` → `sched`	divide by cost	—	—	Cost-awareness is ornament

6.5.2 How Much to Divide by Cost

Figure to be generated

Total validation CPU against the cost exponent c, swept over $[0, 1.25]$, paired per task with bootstrap bands. $c = 0$ is ranking on the rejection rate alone; $c = 1$ is the plain ratio.

Figure 6.3: Cost-exponent sweep. An interior minimum would mean the full cost division over-corrects, which is what a noisy online denominator predicts; a minimum at an endpoint would mean one of the two published positions is already right.

Table 6.11: Hint dose: number of hints crossed with disclosure level. "Leakage" is the share of rejected patches whose rejecting tests are disjoint from what the prompt showed — the patch satisfied everything it was told and died elsewhere.

k	Disclosure	Duplicate rate	`success@B`	Leakage	Prompt tokens
1	`full`	—	—	—	—
2	`full` (default)	—	—	—	—
3	`full`	—	—	—	—
2	`no-required-line`	—	—	—	—
2	`sample-only`	—	—	—	—

Figure to be generated

Dose-response curve: leakage and `success@B` against the amount disclosed, from `sample-only` through `no-required-line` to `full`, at $k \in \{1, 2, 3\}$.

Figure 6.4: The disclosure dose. A curve that is flat across both axes means there is no dose and the disclosure dial does not exist; a leakage rate that climbs with disclosure means the model is beginning to satisfy the tests it was shown rather than repairing the fault.

Table 6.12: Sensitivity configurations, each paired against `sched`. These belong to threats to validity rather than to the contribution: the question is whether RQ1's answer survives choices made for other reasons.

Configuration	What it changes	Δ CPU	95 % CI
`sched-cs-median`	Cold start at the median instead of the maximum	—	—
`sched-cs-q75`	Cold start at the 75th percentile	—	—
`sched-bucket`	Discrete buckets instead of a continuous score	—	—
`sched-explore0`	No exploration ($\varepsilon = 0$)	—	—

6.5.3 The Hint Dose

6.5.4 Sensitivity to Design Choices

6.6 Summary of Findings

To be filled from the 100-task pilot. One short paragraph per research question, stating the answer, the interval, and — where the pilot does not settle the question — saying so plainly rather than reporting a direction that the interval does not support. RQ1 and RQ3 carry Holm-corrected primary contrasts; the ablation ladder and both sweeps are exploratory and are to be described as such.

Chapter 7

Discussion

7.1 When Caster Does Not Help

A method is more useful when its failure conditions are stated than when they are discovered by the next person. CASTER dominates the incumbent only where *three* conditions hold together, and each of the three has a regime in which the method degrades to something already published — or to nothing at all. Table 7.1 states them, and the rest of this section explains each in turn.

Table 7.1: The three conditions under which CASTER is expected to dominate, how each is measured, and what the method collapses to when the condition fails.

Condition	Measured by	What happens when it fails
C1. The test pool is large enough ($\lvert P\rvert \geq 10$)	$\lvert P\rvert$, a property of the frozen pool, known before any run	With a handful of tests there is little to order, and a candidate that survives pays for the whole pool whatever the order. Every policy converges to one schedule.
C2. Test costs are *not* uniform	The within-task coefficient of variation of per-test CPU, from the frozen cost table	$\hat{m}$ becomes a constant, dividing by it changes no ranking, and the scheduler collapses into the rejection-rate policy — FRTP with a denominator.
C3. The task is hard enough for the proposer	The repair rate of the floor configuration on that task, which is an *outcome* and therefore only ever a post-hoc band	A task the model fixes in one or two rounds never builds a history. There is almost no validation to schedule and almost nothing to select hints from, so both mechanisms are inert.

C1 and C2, briefly. The scheduler's gain comes from choosing a good first test out of many, so a pool of three tests has six possible orders and little to choose between them; at $\lvert P\rvert = 1$ every policy is the same schedule by construction. And if every test costs the same, $\hat{m}$ is a constant, dividing by it changes no ranking, and what remains is

FRTP with a denominator — the method degrading gracefully to the published baseline in the regime where that baseline is already correct. The quantity that matters for C2 is the spread *within* a task, not across tasks, because scheduling happens inside a task: a corpus of individually uniform tasks looks heterogeneous in aggregate and offers the scheduler nothing.

7.1.1 C3 — A Task Too Easy for the Proposer

The third condition is the one easiest to overlook, because it is a property of the *pairing* of a task with a model rather than of the task alone.

CASTER only has anything to work with once the loop has failed a few times. Both mechanisms read a history: the scheduler needs runs recorded against tests before K and $\hat{m}$ mean anything, and hints are selected only from tests that have actually rejected a candidate in this session. A task the proposer repairs on round one or two never produces that history. In the limit the comparison is empty — if the first candidate is accepted, every configuration runs the same single scan in the same state, and the measured difference between them is zero by construction, not by failure.

The consequence for reporting is sharper than it looks. A corpus of easy tasks does not make CASTER look bad; it makes every configuration look identical, and it *dilutes* a corpus-wide average towards no effect. Averaged over a corpus where half the tasks are repaired immediately, a real effect on the other half is halved. So difficulty is not a caveat to mention in passing — it is a stratification variable, and an effect reported without it is reporting the mixture rather than the mechanism.

Difficulty is an outcome, so it may never select the corpus. C1 and C2 are properties of the frozen pool: both are known before any candidate is generated, so banding on them is a stratification of a fixed population. C3 is not. The repair rate of a task is produced *by* the experiment, and admitting tasks on the basis of it would be selecting the corpus on an outcome — the corpus would become a function of how the model happened to perform, and no later analysis could undo that. The repair rate used for banding is therefore taken from one designated floor configuration, frozen once beside the corpus checksum, never recomputed per analysis, and computed *per model*, because the same task is a different difficulty for a 7B model and for a hosted one.

7.1.2 How the Three Are Reported

Because all three conditions are bands rather than screens, the pilot reports both views from one run (Section 5.2.3): the whole corpus, and the sub-population where all three

conditions hold. The two numbers answer different questions and both belong in the report.

- **The whole-corpus number** is what a deployment gets if it points the tool at everything, including the tasks the model fixes immediately and the pools too small to order. It is the honest headline for "what does this buy me".
- **The banded number** is what the mechanism does where the mechanism applies. It is the honest answer to "does the idea work", and it is the only one of the two that can be compared against a theory.

Reporting only the first understates a real effect by averaging it against tasks where no effect is possible. Reporting only the second is cherry-picking unless the band boundaries were declared in advance — which is why Section 5.2.3 fixes them before the corpus run rather than after seeing which cut looks best.

7.2 Reporting CPU Rather Than Executions

The prioritisation literature reports the number of test executions rather than elapsed time, and the reason is good: the largest study of patch validation to date counts executions rather than seconds precisely because elapsed time is unstable across environments [22]. Time depends on the machine, on what else is running, and on the interpreter; a count does not.

The problem is that the reproducible unit and the deployed unit are different quantities, and the choice between them is not neutral when test costs vary. A deployment pays for seconds. If a schedule runs more tests but finishes sooner, a paper reporting executions concludes that it lost and a deployment running it finds that it won. The two numbers can rank two schedules in opposite orders, and nothing in the execution count signals when this is happening.

This report therefore reports **both units for every configuration** — it costs nothing, since a loop that measures time is already counting executions — and reports CPU rather than wall clock, which on a machine running several workers is contaminated by siblings. It also reports the **ratio**, CPU per execution, because that is the diagnostic separating "found the rejection in fewer steps" from "found it in cheaper steps": two schedules with the same total CPU and very different ratios are doing different things.

None of this contradicts Lou et al.'s reasoning. Their setting — Java projects with method-level test suites — is also one where test *selection* is available: a test can be skipped entirely because no changed method reaches it, and in that setting they find

selection to dominate prioritisation. In the setting studied here a test is one whole-program execution on one input, so there is no call graph to cut and nothing to select away. Prioritisation is the only lever, and the cost spread inside a pool is what it has to work with.

7.3 Combining with Acceleration and Verifiers

CASTER changes only two things: which test runs first, and what goes into the prompt. It does not change how a test is executed, and it does not change whether a candidate is executed at all. That makes it composable with the other two families of validation-cost reduction, and the composition is worth stating because the three operate on different factors of the same bill.

Each family reduces a different factor of Equation (2.1), and none reduces another family's.

Execution acceleration reduces the third factor. UNIAPR [9] and EXPRESS-APR [10] make each test run cheaper by reusing a warm runtime and by adapting mutation-testing techniques, reporting up to 137× over plain validation. A scheduler is indifferent to how a test is executed: it ranks by measured cost, so if acceleration changes the costs, the ranking simply re-learns them. One interaction is worth noting, and it cuts in a useful direction: acceleration typically removes a fixed per-run overhead, which *increases* the relative spread between cheap and expensive tests and therefore gives the cost term more to work with rather than less.

Execution-free verification reduces the first factor. R2E-GYM [11] finds learned verifiers and test-based verifiers to be complementary — test-based verifiers often cannot distinguish among candidates, while learned verifiers are biased towards stylistic features, and combining them beats either alone. A learned verifier placed before validation filters candidates cheaply; the scheduler then decides how to spend execution on those that survive. The two do not compete for the same decision.

Selected hints also reduce the first factor, but earlier and by a different route: rather than filtering candidates after they are generated, they change what is generated. The two are compatible, and a system using both would pay the generation cost only for candidates that neither the hint contract nor the verifier has reason to reject.

The practical reading is that these are not alternatives to be chosen between. A deployment that cares about validation cost should expect to run an accelerated executor, a scheduler on top of it, and a filter in front of both; this work supplies the

middle layer and the generation-side half, and is explicitly designed not to constrain the other two.

7.4 The Cost of Adopting It

Section 7.3 argued that the three families of validation-cost reduction compose. They do not, however, cost the same to adopt, and for a team deciding what to do first that difference matters as much as the size of the saving.

Execution acceleration [9, 10] needs an integration with one language runtime — class reloading, state resetting — and the correctness argument that comes with reusing a dirty runtime: a large payoff, tied to a specific stack. Execution-free verification [11] needs training data, a training run and a second model to serve, plus a standing question about distribution shift. Search over repair attempts [13] spends extra model calls per bug, a direct increase in the resource this work is trying to reduce. Caster needs three counters, one sort per round, and a timer around a call the loop already makes (Table 4.2).

Scheduling is therefore the cheapest of the four to try and the easiest to withdraw: if it does not help on a given corpus it degrades to the ordering the loop already used, and removing it is deleting a comparator. Nothing about the rest of the system has to change to accommodate it, and nothing has to be retrained when the proposer model is swapped — the statistics are of the task and the session, not of the model, so a new model simply starts writing a new record.

This is also an argument about where a cost-reduction effort should start. Acceleration and verifiers both have larger ceilings; they also both require a project. A scheduler is an afternoon, and it establishes whether the pool has any cost structure worth exploiting before anyone commits to the larger work. If the within-task cost spread turns out to be flat (Section 7.1, C2), that is worth knowing *before* building a runtime integration, not after.

Simplicity is a claim that has to survive the ablation. A cheap mechanism is only a virtue if it is doing something. The ablation ladder of Section 5.5 exists partly for this reason: if the whole effect is recovered by `kill-only`, then the cost term is ornament and the honest report is that Frtp with a denominator was enough. The point of the present section is not that Caster is good because it is simple, but that *if* the effect survives the ablation, it was bought very cheaply.

7.5 What the Pilot Can and Cannot Settle

A pilot of 100 tasks is a screening instrument. It can establish that an effect exists and is large enough to justify a corpus-scale run, it can rule out directions whose intervals straddle zero by a wide margin, and it can fix the parameters that the full run must declare in advance. It cannot establish a magnitude: an effect size estimated on 100 tasks and then reported as the headline of a corpus-scale study is a selected maximum, and the honest handling is to declare the parameters now and report the full-corpus number as the result.

One specific caution applies to the cost exponent c and the hint count k. Both are chosen by a sweep on the pilot data. If the full run then reports results at those values without saying so, the reported effect inherits the selection. Section 9.2 therefore commits to declaring both before the corpus run and to reporting the sweeps as sensitivity rather than as findings.

Chapter 8

Threats to Validity

8.1 Internal Validity

Parameters are selected on the pilot. The cost exponent c, the hint count k, the exploration rate ε and the cold-start rule are all chosen by sweeps on the same 100 tasks the results are reported on, and a value selected on the data it is reported on is optimistically biased. The mitigation is procedural rather than statistical: all four are declared before the corpus run, and the sweeps are then reported as sensitivity checks against the declared values (Section 9.2).

The history store depends on the patches the model happens to produce. Policy and data are coupled within a session: a schedule influences which tests are run, which influences what the store contains, which influences the next schedule. Two things limit the damage. Round 1 is identical across configurations under common random numbers, so no configuration starts from an advantage; and the per-test cost table is established by a dedicated full-scan audit before any configuration runs, so no configuration's ordering can contaminate the costs every configuration is scored against. The combined `sched+hints` configuration has the same issue by design — both mechanisms read one store — which is why the 2×2 design of Table 5.6 also runs each mechanism alone.

Instrument faults that produce numbers rather than errors. Three failure modes would yield plausible wrong values instead of a crash: a (patch, test) pair executed and counted twice, a cached outcome disagreeing with a fresh one, and a timeout entered into the cost model at the clock value. Each is instrumented — an accounting identity checked per scan, a halt on disagreement, and quarantine of timeouts — rather than asserted in prose.

8.2 External Validity

One benchmark, one language. All results come from the Python subset of ConDefects: single-file competitive-programming submissions with whole-program, stdout-compared tests. Two consequences follow. Test *selection* is unavailable here, so prioritisation is the only scheduling lever, whereas in project-scale Java repair selection is available and has been found to dominate [22]. And contest pools are deliberately heterogeneous in cost, which is favourable to the cost term; an interpreted language exaggerates that spread further. A uniform-cost corpus would show less, which is the regime Section 7.1 identifies as C2.

One proposer model, at pilot scale. Every candidate comes from `qwen2.5-coder:7b` at a single temperature, so the rejection structure the store learns is one model's: which tests are discriminating depends on which mistakes the model makes. Whether that structure transfers across models is an open question and a planned follow-up (Section 9.2). The budget of twenty rounds also truncates the learning curve — the cost estimates are still improving when a session ends — so a longer budget could show a larger or a smaller effect, and the pilot cannot say which.

8.3 Construct Validity

"Duplicate" is defined by normalised text. Two candidates that differ in a variable name but express the same repair hypothesis are not duplicates under this definition, although they are equally wasteful. The measured duplicate rate is therefore a *lower bound* on redundancy, and a reduction in it is a conservative estimate. The same caveat applies to the 65 % figure reported for ChatRepair [7], which this work uses as its reference point.

Cost is CPU; acceptance is plausibility. A deployment paying for wall clock on a loaded machine, or for a test service priced per invocation, would prefer a different schedule from one paying for CPU — and the units can disagree (Section 7.2), which is why both CPU and executions are reported for every configuration. Separately, a candidate surviving the whole pool is *accepted*, and the pool is finite: passing a finite suite is weaker than being correct [21], so `success@B` is a plausibility rate. That does not affect the cost claims, which hold the acceptance criterion fixed across configurations, but it bounds what the repair-rate numbers mean.

The sandbox clock shapes the corpus silently. A wall-clock kill is applied to the reference program first, so a test the correct program cannot finish leaves the pool, and a task whose pool empties leaves the corpus. This keeps the oracle sound — no correct patch is rejected by a test the reference also fails — but it removes faults non-randomly, from the expensive end. The admission funnel (Table 6.1) counts how many go this way, so the bias is visible rather than silent.

8.4 Conclusion Validity

Sample size and skew. One hundred tasks with three seeds is small for an effect whose per-task distribution is heavily right-skewed and dominated by a few expensive tasks. Mean-based tests would be driven by those tasks, so comparisons use the Wilcoxon signed-rank test on per-task totals; intervals are reported for every contrast, no point estimate appears without one, and every aggregate is reported alongside the number of tasks going each way.

Clustering. The three seeds on one task share the task, the pool and the cost structure, so treating them as independent would understate every interval by roughly $\sqrt{3}$. Intervals therefore come from a cluster bootstrap that resamples whole tasks.

Multiple comparisons. The experiment produces many contrasts: five configurations in RQ1, four in RQ2, a four-cell factorial, a five-step ablation ladder, two sweeps and a sensitivity table. Testing all at one threshold would make a spurious result close to certain, so four primary contrasts are declared in advance and carry a Holm correction (Section 5.7). Everything else is labelled exploratory, and no conclusion rests on a single exploratory row.

Chapter 9

Conclusion and Future Work

9.1 Conclusion

An LLM-based repair loop produces useful information on every iteration and then discards almost all of it. Every validation reveals which test rejected which candidate and what that test cost; every generation reveals which repair idea has already been tried. Current systems keep neither in a form that the next iteration can use: schedulers rank tests by a count of past rejections and ignore how long a test takes, and prompts either carry nothing or carry a transcript that grows without bound.

This report described Caster, which keeps that information in one small record per test case — how often the test has run, how often it rejected a candidate, and how much CPU time those runs took — and spends it twice. **Cost-aware validation scheduling** ranks tests by an estimated rejection rate divided by an estimated cost, so an unpromising patch is rejected as cheaply as possible rather than merely in as few steps as possible. **Selected hints** replace the accumulated transcript with a small, freshly chosen set of concrete requirements, so the prompt stays a constant size while its content changes to match what the session has just learned. The two mechanisms share one record by construction, which is what makes them a system rather than two features: scheduling produces the rejection evidence that hints are selected from, and better hints send fewer wasted candidates to the step that costs the most.

The whole mechanism is one record of three numbers per test case, updated in constant time. Nothing is trained, no profiling pass is required, and neither half adds a program execution or a model call: the scheduler reorders runs the loop was already making, and hint selection only reads what validation has already written down. That matters for more than tidiness — the two other families of validation-cost reduction each require a project before they return anything, where this requires a counter and a timer.

Because the approach operates only on which test runs first and what goes into the prompt, it does not compete with the other two families of validation-cost reduction. It composes with execution acceleration [9, 10], which makes each run cheaper, and

with execution-free verifiers [11], which remove candidates before they are run at all.

The evaluation is a pilot on 100 ConDefects Python tasks, laid out in Chapter 5 and reported in Chapter 6. Its purpose is to establish whether the effects exist at a scale worth a full corpus run and to fix the parameters that the full run must declare in advance. It is not a corpus-scale result and is not reported as one.

9.2 Future Work

1. **Run the full ConDefects corpus (roughly 1,244 faults).** The pilot screens; the corpus measures. Three things change at that scale. The admission screen admits every sound fault rather than a drawn subset, so the corpus is no longer correlated with whatever finished first. Banding by pool size and by contest era becomes possible, which is what separates "the method works" from "the method works on large heterogeneous pools", the distinction Section 7.1 says is the real one. And the parameters chosen on the pilot — the cost exponent c, the hint count k, the exploration rate and the cold-start rule — are declared *before* the run and reported as sensitivity afterwards, so the headline is not a selected maximum.

2. **Add a benchmark with method-level test suites, such as Defects4J [23].** Every test in ConDefects is a whole-program execution, so nothing can be *selected away* and prioritisation is the only lever. A project-scale Java benchmark has a call graph, and therefore has selection — which is the setting where Lou et al. [22] find selection to dominate prioritisation. Two questions follow. Does a cost-aware schedule still help once the cheap tests have already been selected away? And is the right design a scheduler layered on top of a selector, where selection decides the candidate set and the cost-aware rank decides the order inside it? That setting is also where the method's own failure condition can be tested directly: method-level suites are more uniform in cost than contest pools, so the cost term should matter less there, and a measurement that shows otherwise would mean the mechanism is not the one claimed.

3. **Add a second proposer model to test whether the rejection structure is model-dependent.** Everything the store learns is a statistic over the candidates one model produced. If two different models make different mistakes, different tests are discriminating for each, and the scheduler would re-learn per model — which is fine, since it learns online, but it would mean the reported effect size is one model's. If instead the same tests are discriminating across models, the rejection structure is a property of the *task* rather than of the proposer, which is a stronger and more

useful claim: it would mean a cost table and a rejection profile could be computed once per benchmark and reused.

4. **Report both bills together, and measure redundancy at the hypothesis level.** Validation CPU and generation tokens are halves of one cost, and optimising either alone can inflate the other, so the pair should be reported rather than either on its own. Separately, the duplicate rate used here is defined over normalised patch text and undercounts: two textually different patches expressing the same repair idea are equally wasteful but are not duplicates by that definition. Clustering candidates by repair hypothesis would give a tighter bound on how much redundancy a steering mechanism actually removes.

References

[1] Yonghao Wu, Zheng Li, Jie M. Zhang, and Yong Liu. ConDefects: A complementary dataset to address the data leakage concern for LLM-based fault localization and program repair. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE)*, pages 642–646, 2024. doi: 10.1145/3663529.3663815.

[2] Yuhua Qi, Xiaoguang Mao, and Yan Lei. Efficient automated program repair through fault-recorded testing prioritization. In *Proceedings of the 29th IEEE International Conference on Software Maintenance (ICSM)*, pages 180–189, Eindhoven, The Netherlands, 2013. doi: 10.1109/ICSM.2013.29.

[3] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. Automated program repair. *Communications of the ACM*, 62(12):56–65, 2019. doi: 10.1145/3318162.

[4] Zhihao Lin, Junhua Zhu, Mingyi Zhou, Xin Wang, Zhensu Sun, Renyu Yang, David Lo, and Li Li. To run or not to run: Analyzing the cost-effectiveness of code execution in LLM-based program repair. In *Proceedings of the ACM on Software Engineering (ISSTA)*, 2026. To appear. arXiv:2606.26978.

[5] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. RepairAgent: An autonomous, LLM-based agent for program repair. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 2188–2200, Ottawa, ON, Canada, 2025. doi: 10.1109/ICSE55347.2025.00157.

[6] Chunqiu Steven Xia and Lingming Zhang. Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 819–831, Vienna, Austria, 2024. doi: 10.1145/3650212.3680323.

[7] Aolin Chen, Haojun Wu, Qi Xin, Steven P. Reiss, and Jifeng Xuan. Studying and understanding the effectiveness and failures of conversational LLM-based repair. *arXiv preprint arXiv:2503.15050*, 2025.

[8] Cuong Chi Le, Minh Le-Anh, Cuong Duc Van, and Tien N. Nguyen. Semantic

evolution over populations for LLM-guided automated program repair. *arXiv preprint arXiv:2604.02134*, 2026.

[9] Lingchao Chen, Yicheng Ouyang, and Lingming Zhang. Fast and precise on-the-fly patch validation for all. In *Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 1123–1134, Madrid, Spain, 2021. doi: 10.1109/ICSE43902.2021.00104.

[10] Yuan-An Xiao, Chenyang Yang, Bo Wang, and Yingfei Xiong. Accelerating patch validation for program repair with interception-based execution scheduling. *IEEE Transactions on Software Engineering*, 50(3):618–635, 2024. doi: 10.1109/TSE.2024.3359969. arXiv:2305.03955.

[11] Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2E-Gym: Procedural environments and hybrid verifiers for scaling open-weights SWE agents. In *Proceedings of the 39th Conference on Neural Information Processing Systems (NeurIPS)*, San Diego, CA, USA, 2025. arXiv:2504.07164.

[12] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824, 2025. doi: 10.1145/3715754.

[13] Antonis Antoniades, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. SWE-Search: Enhancing software agents with monte carlo tree search and iterative refinement. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, Singapore, 2025. arXiv:2410.20285.

[14] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, pages 8634–8652, New Orleans, LA, USA, 2023.

[15] Sungmin Kang, Bei Chen, Shin Yoo, and Jian-Guang Lou. Explainable automated debugging via large language model-driven scientific debugging. *Empirical Software Engineering*, 30(2), 2025. doi: 10.1007/s10664-024-10594-x.

[16] Xin Yin, Chao Ni, Shaohua Wang, Zhenhao Li, Limin Zeng, and Xiaohu Yang. ThinkRepair: Self-directed automated program repair. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 1274–1286, Vienna, Austria, 2024. doi: 10.1145/3650212.3680359.

[17] Silin Chen, Shaoxin Lin, Yuling Shi, Heng Lian, Xiaodong Gu, Longfei Yun, Dong Chen, Lin Cao, Jiyang Liu, Nu Xia, and Qianxiang Wang. SWE-Exp: Experience-driven software issue resolution. *arXiv preprint arXiv:2507.23361*, 2025.

[18] Nikhil Parasaram, Huijie Yan, Boyu Yang, Zineb Flahy, Abriele Qudsi, Damian Ziaber, Earl T. Barr, and Sergey Mechtaev. The fact selection problem in LLM-based program repair. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 2574–2586, Ottawa, ON, Canada, 2025. doi: 10.1109/ICSE55347.2025.00162.

[19] Sebastian Elbaum, Alexey Malishevsky, and Gregg Rothermel. Incorporating varying test costs and fault severities into test case prioritization. In *Proceedings of the 23rd International Conference on Software Engineering (ICSE)*, pages 329–338, Toronto, ON, Canada, 2001. doi: 10.1109/ICSE.2001.919106.

[20] Gregg Rothermel, Roland H. Untch, Chengyun Chu, and Mary Jean Harrold. Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering*, 27(10):929–948, 2001. doi: 10.1109/32.962562.

[21] Zichao Qi, Fan Long, Sara Achour, and Martin Rinard. An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 24–36, 2015. doi: 10.1145/2771783.2771791.

[22] Yiling Lou, Jun Yang, Samuel Benton, Dan Hao, Lin Tan, Zhenpeng Chen, Lu Zhang, and Lingming Zhang. When automated program repair meets regression testing—an extensive study on two million patches. *ACM Transactions on Software Engineering and Methodology*, 33(7), 2024. doi: 10.1145/3672450.

[23] René Just, Darioush Jalali, and Michael D. Ernst. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 23rd International Symposium on Software Testing and Analysis (ISSTA)*, pages 437–440, 2014. doi: 10.1145/2610384.2628055.

[24] Harold Jeffreys. An invariant form for the prior probability in estimation problems. *Proceedings of the Royal Society of London. Series A*, 186(1007):453–461, 1946. doi: 10.1098/rspa.1946.0056.